

生产过程中的决策问题

摘要

本文针对生产过程中的决策问题，建立了分层随机抽样检测模型，给出了最优抽样检测方案；建立了多阶段决策模型，给出了各个情况的具体最优决策方案；建立了多工序多阶段决策模型，给出了多工序下的最优决策方案；建立了抽样检测仿真模型，给出了问题一和问题二的最优决策方案。

针对问题一，建立了分层抽样检测模型，给出了90%和95%的信度下的最少抽样检测次数方案。首先，在稳定条件下，考虑到每个零配件的检测结果是独立的且次品率保持一致，建立了二项分布模型来拟合分层抽样检测过程。然后，在此基础上，建立了假设检验模型，全面评估了零配件的可靠性。接着，运用MATLAB对建立的分层抽样检测模型进行求解，在不同的允许误差下，确定了相应的最优检测方案。例如，当允许误差为0.05时，在95%的信度下最少检测次数为98，在90%的信度下最少检测次数为59。最后，对最少检测次数进行灵敏度分析，发现最少检测次数对标称值不敏感，表明模型的稳健程度较高。

针对问题二，建立了多阶段决策模型，给出了各个情况的最优决策方案以及决策依据和指标结果。首先，将企业生产各个阶段进行可视化处理，让整个生产过程清晰明了。然后，以0-1整数规划模型为基础，建立了多阶段决策模型，对各个阶段进行精细化分析，推导出了各个阶段的合格率、次品率和成本的数学表达式，确定了总成本的表达式。接着，运用穷举法对决策路径进行全面搜索，以总成本最小为目标，确定了各情况下的最优决策方案，并运用控制变量法确定每个阶段的决策依据。例如，情况1的最优决策方案为方案16，总花费成本为2380元，其他情况具体见表6。最后，对模型进行蒙特卡洛检验，对比分析得出，添加随机性次品率变化浮动较小，且比未添加随机性次品率更契合实际，表明本题的决策方案效果较好。

针对问题三，建立了多工序多阶段决策模型，给出了多工序下的最优决策方案以及决策依据和指标结果。首先，与问题二类似，以0-1整数规划模型为基础，建立了子工序规划模型，推导出了三个零部件情况下的各阶段合格率、次品率和成本的数学表达式。然后，结合问题二的结论，确定了总成本的表达式。接着，运用穷举法进行求解，确定了最优决策方案为：全部零部件和半成品均未检测，并运用控制变量法确定了每个阶段的决策依据。最后，对总成本进行扰动检验，发现次品率几乎不对总成本造成扰动，说明本题建立的模型鲁棒性较高。

针对问题四，建立了抽样检测仿真模型，更新了问题二和问题三的最优决策方案。首先，在问题一的二项分布基础上，运用贝叶斯理论，推导出了零配件、成品和半成品的后验函数。然后，以混合整数规划模型为基础，结合问题一和问题二的成本表达式，推导出了总成本的数学表达式。接着，运用鲸鱼算法，结合割平面法和分支定界法对模型进行求解，以问题二和问题三所确定的决策依据为支撑，确定了问题二和问题三更新后的最优方案，例如：问题二中情况1的最优决策方案为：零部件均不检测、检测成品、拆解不合格成品；问题三的最优决策方案为：全部零部件和半成品均未检测，半成品1和2未拆解，半成品3拆解，成品检测，成品未拆解。最后，运用对比分析法计算出了鲸鱼算法的最优参数，增强了结果的客观性。

关键词：二项分布 0-1整数规划 穷举法 混合整数规划 鲸鱼算法

一、问题重述

1.1 问题背景

当前，电子产品种类众多，极大地便利了人类生产、生活及工作的各个方面。尤其是在人工智能发展迅速的现在，众多企业竞相涌入电子产品制造行业，导致该领域的竞争愈发激烈，形势渐入白热化阶段^[1]。对此，本文深入剖析了某企业生产流程中的具体问题，并据此制定出一系列切实可行的生产决策方案，旨在助力该企业在激烈的市场竞争中脱颖而出。

某企业购买两种零配件，用于装配和生产畅销的电子产品。为了提高市场效益、生产出畅销市场的优质产品，企业制定了一系列合理的方案。具体来说，在成品质量评估方面，主要聚焦于零配件的合格性。如果成品中发现任何一个零配件不合格，那么整个成品将被直接判定为不合格；而即便两个零配件均符合标准，成品也未必能达到合格标准，还需经过全面的质量检测。针对不合格产品的处理，企业采取了两种主要策略：报废与成品拆解。报废是一种直接且迅速的处理方式，能够迅速清除不良库存。而成品拆解则是一种更为细致的方法，它允许企业对次品进行逐一分析，从而降低生产成本，为产品的组装提供了更多的合格零部件。值得注意的是，成品拆解过程对零部件本身的质量并无任何负面影响，但执行此流程需承担一定的费用成本。尽管如此，这两种方法均为企业有效管理不合格产品、提升整体产品质量提供了有力支持。

1.2 问题重述

针对上述生产流程中的多决策问题，通过查阅相关文献了解了专业背景，根据题目的具体要求，建立了数学模型，解决了如下问题：

对于问题一，为了证明零配件的次品率没有超过标称值，企业运用抽样检测的方法进行验证并且决定是否接收这批零件。在此流程中，企业需要承担抽样检测的相关费用。在此情况下，确定企业检测次数最少的检测方案。

当标称值在10%时，根据确定的最优检测方案，针对以下两种情况，建立数学模型，运用合适的算法，求解出具体结果：

(1) 在95%的信度下，当检测结果的次品率高于标称值时，企业拒收这批零部件。

(2) 在90%的信度下，当检测结果的次品率低于标称值时，企业接收这批零部件。

对于问题二，在已知零配件和成品次品率的情况下，对企业生产流程的各个阶段做出决策。企业决策流程主要包括零配件检测、成品检测、不合格成品的拆解以及用户购买不合格品调换的问题。最后，结合表1中企业在生产中遇到的情况，给出具体的决策方案、依据以及相应的指标结果。

对于问题三，在问题二的基础上，在已知零部件、半成品和成品的次品率的情况下，确定 m 道工序、 n 个零配件的生产流程的决策方案。结合题目中给出的两道工序、8个零配件的组装和生产中遇到的具体情况，给出具体的决策方案、依据以及相应指标。

对于问题四，运用问题一中的抽样检测方法预测问题2和问题3的零部件、半成品和成品的次品率，在此基础上，重新确定决策方案、依据和相应的指标。

二、问题分析

2.1 问题一的分析

据基本信息可知，为了确保所生产的畅销电子产品质量得以保障，一家企业在采

购其所需的两类关键零配件前，决定对供应商提供的零配件实施严格的质量检验。尽管供应商已预先提供了零配件的次品率数据，但为防范潜在的数据不准确风险，从而避免企业利润受损，企业决定自主进行抽样检测。考虑到检测费用完全由企业承担，因此要求设计一个检测次数尽可能少的抽样检测方案。鉴于供应商声明的最大次品率为10%，本题建立以分层抽样检测为基础的抽样检测模型。以标称值为检测基准，明确设定接收与拒绝的信度标准，以此约束检测样本范围，在确保检测结果的准确性和可靠性的前提下，减少检测次数，获得最代表的样本数据。

2.2 问题二的分析

本题属于决策类问题，解决该类问题主要运用穷举法、多属性决策等多种方法。针对本题的具体条件和决策要求，本题决定以0-1整数规划模型为基础进行建模。在该模型深入分析从零配件到成品检测的全过程中，利用0-1整数规划的特性，对检测决策进行精细化分析。根据所涉及的成本结构以及不合格品出现的概率，逐步推导出总成本的精确表达式。为了找到全局最优的决策方案，运用穷举法对所有可能的决策路径进行全面搜索。在此基础上，运用控制变量法反推决策依据。最后，运用蒙特卡洛算法对模型进行检验。在`calc_total_cost`函数中添加随机性，运用蒙特卡洛模拟估计次品率的值，对比添加随机后的次品率变化情况，进一步验证最优方案选择的准确性。

2.3 问题三的分析

针对问题三，在问题二的基础上增添了一个更复杂的生产流程，涉及 m 道工序和 n 个零配件。本题要求在已知各零配件、半成品及成品次品率未知的情况下，为整个生产过程制定最优的决策方案。首先，需要先分析各道工序之间的关系，以及零配件、半成品和成品的次品率是对最终的产品质量的影响，以确保在制定决策时能够全面考虑各个环节的相互影响因素。其次，根据不同的情况设计多种可能的检测策略组合(各零配件、半成品及成品的检测决策等)。决策组将直接影响生产成本、次品率及最终的市场竞争力。通过生成所有可能的策略组合，再利用数学模型评估每种策略的优劣，综合考虑购买单价、检测成本、装配成本、不合格产品的调换损失以及市场售价等因素，选出最优的策略。最终，为企业生产活动提供具体的决策方案，帮助企业在实际生产情况下实现效率和质量的双重优化。

2.4 问题四的分析

本题在问题二和问题三的基础上，改变次品率的计算方法为抽样检测方法。据于此，引如二项分布下的叶贝斯更新，对次品率进行更新仿真。接着，将更新后的次品率带入问题二和问题三得出的各个阶段和总成本的数学表达式中，建立混合整数规划模型。对混合整数规划模型求解算法主要包括精确算法和启发式算法，如分支定界法、粒子群法、模拟退火算法等等。本文选用基于分支定界法和割平面法的鲸鱼算法进行求解，依据以及相应的指标，确定问题二中的各个情况的最优决策方案以及问题三的最优决策方案。最后，运用对比分析验证，确定了鲸鱼算法中最优的参数设定，增强了结果的客观性。

三、模型假设

- 1.假设每次抽样检测的结果是独立的，即每次检测的结果不会影响其他检测的结果；
- 2.假设生产流程中的次品随机出现；
- 3.假设分层抽样法抽到的次品概率相等；
- 4.假设装配流程中随机选择零配件，即零配件的选择不以人的意志为转移；

四、符号说明

符号	含义	单位
L_{lj}	状态 j 的次品数	个
G_{lj}	状态 j 的合格品数	个
m_j	状态 j 的零配件数量	个
W	决策过程产生的总成本	元
$COST_3$	成品检测成本	元
$COST_4$	单个不合格品拆解成本	元
$P(D)$	边缘概率	
$C(n,k)$	从 n 个不同项目中选择 k 个方法数	
$X(t)$	鲸鱼的具体位置	

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型建立的准备

由题意得，本题目标是设计一个高效且经济的检测方案，旨在减少检测次数，从而控制成本，避免对利润造成不必要的影响。通过查阅相关资料可知，抽样方法^[2]主要分为随机抽样法、分层抽样法、整群抽样法以及系统抽样法。考虑到供应商已提供了次品率的标称值，在稳定条件下每个零配件的检测结果是独立的且次品率保持一致，满足二项分布的条件，因此本题决定以二项分布模型为基础运用分层抽样检测法进行模拟。

同时，考虑到检测样本量大且次品率 p 不处于极端值(即不接近0或1)，次品数分布可以合理地近似为正态分布。这一特性使我们能够利用中心极限定理，对次品率进行更为精准的统计分析。此外，为了全面评估供应商数据的可靠性，本题引入了假设检验模型，通过设定两个信度阈值(90%和95%)，分别对应接收和拒收条件，对检测样本数量进行了严格的约束和规范。

5.1.2 模型的建立

5.1.2.1 分层抽样检测模型

5.1.2.1.1 二项分布模型

由于零配件次品检测的本质为概率事件，本题运用了分层抽样^[4]检测法来优化样本检验过程。考虑到每个受检零配件均作为独立个体存在，其检测结果仅呈现为“合格”或“不合格”两种明确状态，且这一结果对其他零配件的检测无任何干涉效应。根据供应商所提供的标称值，可以合理推断，在保持检测条件一致的前提下，每次检测中次品的出现概率是恒定的，这一特性恰好契合了二项分布的基本假设。因此，本题选择运用二项分布^[3]来对零配件的检测过程进行精确模拟与分析。

二项分布 $B \sim (n, p)$ ，有两个参数， n 表示独立检测的次数， p 表示单次检测出次品的概率。若检测样本容量为 n ，随机变量 x 的分布律为：

$$P(x=k) = C_n^k p^k (1-p)^{n-k}, k=0,1,2,\dots,n \quad (1)$$

称随机变量 x 遵循参数为 n , p 的二项分布, 记作:

$$X \sim N(np, np(1-p)) \quad (2)$$

检测样品中次品数满足 x 的期望值:

$$E(x) = np \quad (3)$$

鉴于企业的大规模进购需求, 导致检测样本基数庞大。根据供应商给出的标称值, 次品率 p 并未趋近于0或1的极端情况, 因此可以合理的运用中心极限定理, 将次品在检测样本中的分布近似看作正态分布。在此基础上, 通过运用正态分布的标准化公式, 可进一步推导出以下表达式:

$$Z = \frac{X - np_0}{\sqrt{np_0(1-p_0)}} \quad (4)$$

式中, p_0 为供应商给出的标称值; Z 为随机变量 X 经过列维-林德伯格中心极限定理的变形后, 在标准正态分布下的新变量, 在数量上表示为标准差 σ 的倍数, 该值可通过 Z 值分布表查得。

5.1.2.1.2假设检验模型(hypothesis testing)

企业针对零配件的验收设定了明确的标准, 旨在通过实际检测所得的次品率与供应商提供的标称值进行对比, 以验证标称值的可信度。具体而言, 当可信度为90%时, 企业将接受这批零配件; 反之, 若检测得次品率远高于标称值, 且这一差异的可信度达到95%时, 表明标称值存在显著不可信的情况, 企业将拒收这批零配件。对此, 本题建立了假设检验模型, 作为衡量是否接受或拒收供应商零配件的标准化依据^[5]。

5.1.2.1.2.195%信度时高于标称值(拒收)

首先, 针对拒收这批零配件这一情况进行假设。为了验证这一假设, 从这批零配件中抽取一部分作为样本数据进行检验。在假设检验的过程中, 通常会设计两个假设: 原假设 H_0 (又称零假设), 一般是统计者倾向于拒绝的假设; 以及备择假设 H_1 , 它则是统计者倾向于接受的假设。由题意可知, 该企业期望供货商提供的零配件次品率尽可能地低。因此, 在本题中, 选择“次品率不高于标称值”(即不符合拒收条件)作为原假设, 而选择“测得次品率超过标称值”(即符合拒收条件)作为备择假设。可表示为:

$$\begin{cases} H_0 = p \leq 10\% \\ H_1 = p > 10\% \end{cases} \quad (5)$$

在随机抽样的过程中, 由于样本的随机性, 检验结果可能会受到小概率事件的影响, 从而导致两种类型的错误: “拒真”错误与“纳伪”错误。具体而言, “拒真”错误, 又称第I类错误, 发生在原假设实际为真的情况下, 然而基于样本数据对总体的估计却导致了原假设被拒绝; 相对地, “纳伪”错误, 或称第II类错误, 则发生在原假设实际为假的情况下, 但依据样本数据对总体的推断却错误地接受了原假设。

若企业意图拒绝的假设(即 H_0)出现错误, 这仅会导致第I类错误的发生。由于第I类错误的概率已通过预设的显著性水平得到了有效控制, 因此能够最大限度地限制原假设错误的情况。显著性水平的高低直接影响了原假设被推翻的难易程度: 显著性水平越低, 意味着原假设越难以被否定, 此时假设检验更为保守; 相反, 显著性水平越高, 则原假设越容易被拒绝, 但这也相应地提高了假设检验的犯错率。当发生小概率事件, 即 $p \leq \alpha$ 时, 这表明企业在对零配件进行抽样检测时样本出现了偏差, 从而判定假设存在错误, 并据此拒绝原假设 H_0 。在此基础上, 可以得出标准化变量的表达式为:

$$Z = \frac{p - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}} \quad (6)$$

式中 Z 为 Z value常量，通过代入置信度可在 Z 值表中查找得 $Z=1.96$ ； p 为测得零配件的次品率。

拒收条件为：

$$Z \geq 1.96 \quad (7)$$

代入表达式(6)可得关于 p 的表达式：

$$p \leq p_0 + 1.96 \times \sqrt{\frac{p_0(1-p_0)}{n}} \quad (8)$$

接着，将供货商提供的标称值 $p_0=10\%$ 代入：

$$p \leq 0.10 + 1.96 \times \sqrt{\frac{0.10 \times 0.90}{n}} \quad (9)$$

即：

$$p \leq 0.10 + \frac{0.588}{\sqrt{n}} \quad (10)$$

通过企业测得的次品率可以求出样本容量 n 与拒收条件的关系。

5.1.2.1.2.290%信度时低于标称值(接收)

具体分析情况与信度为95%时相同，设次品率高于标称值这一不符合接收条件的情况为原假设 H_0 ，设次品率低于标称值这一符合接收条件的情况为备择假设 H_1 ，可得表达式如下：

$$\begin{cases} H_0 = p \geq 10\% \\ H_1 = p \leq 10\% \end{cases} \quad (11)$$

通过代入置信度可在 Z 值表中查找得 $Z=1.645$ ； p 为测得零配件的次品率。

拒收条件为：

$$Z \leq 1.645 \quad (12)$$

代入表达式(1)可得关于 p 的表达式：

$$p \leq p_0 + 1.645 \times \sqrt{\frac{p_0(1-p_0)}{n}} \quad (13)$$

接着，将供货商提供的标称值 $p_0=10\%$ 代入：

$$p \leq 0.10 + 1.645 \times \sqrt{\frac{0.10 \times 0.90}{n}} \quad (14)$$

即：

$$p \leq 0.10 + \frac{0.493}{\sqrt{n}} \quad (15)$$

通过企业测得的次品率即可求出样本容量 n 与接收条件的关系。

为了满足在两种信度条件下的拒收与接收标准，本题采用双侧检验来计算所需的最小抽样量。又因为次品率遵循二项分布，并且题目已给出显著性水平，可用于研究不同拒绝域设置对同一检验方法效果的影响，同时在确保第 I 类错误得到有效控制的前提下，提升检验的整体功效。

针对本题的参数假设检验模型，原假设为 H_0 ，备择假设为 H_1 ，引入某个检验统计量 T 。对于给定的样本容量 n ，将样本空间划分为拒绝域 W 和接受域 S ，且接受域和拒绝

域为两个互不相交的集合。在企业对零配件进行检测的过程中，根据实测的次品率来判断样本是落入拒绝域还是接受域。具体而言，如果样本落入拒绝域，则拒绝原假设 H_0 ；反之，则接受原假设。

本模型依据小概率事件发生的概率，在已确定的显著性水平 α 的基础上，采用双侧等尾拒绝域法。这种方法通过对原假设 H_0 下的分布函数进行分析，来确定拒绝域的临界值：

$$W_1 = \{(x_1, \dots, x_n) : T < c_1 \mid T < c_2\} \quad (16)$$

式中， c_1, c_2 满足 $c_1 < c_2$ ，通过下述表达式可测得临界值：

$$\begin{cases} c_1 = \max \left\{ x : P(x) \leq \frac{\alpha}{2} \right\} \\ c_2 = \min \left\{ x : P(x) \geq 1 - \frac{\alpha}{2} \right\} \end{cases} \quad (17)$$

由于样品次品率满足正态分布函数 $P(x)$ ，对于 $P(x)$ 对应的次品率分布函数 $f(x)$ ，可测得在信度分别为95%和90%下最优的置信度区间：

$$\begin{cases} \int_{c_1}^{c_2} f(x) dx = 1 - \alpha \\ f(c_1) = f(c_2) \end{cases} \quad (18)$$

5.1.3 模型的求解

在模型求解过程中，发现设置不同的误差，最少检测次数存在较大的差异。据于此，本文以最常用的误差0.05开始，以0.01为步长，逐渐递减为0.01，探究不同误差情况下的零部件最少检验次数，为企业生产提供了较为完备的选择，具体如下表所示：

表 1 不同误差下的最少检测次数

信度	误差	最少检测次数
95%	0.01	2436
	0.02	609
	0.03	271
	0.04	153
	0.05	98
90%	0.01	1475
	0.02	369
	0.03	164
	0.04	93
	0.05	59

代入不同假设检验条件下的信度，以及供应商已给出的标称值对企业抽取的样品次品率进行计算，明确置信区域的临界值，最后得出不同信度下的拒收区域和接受区域，如下图：

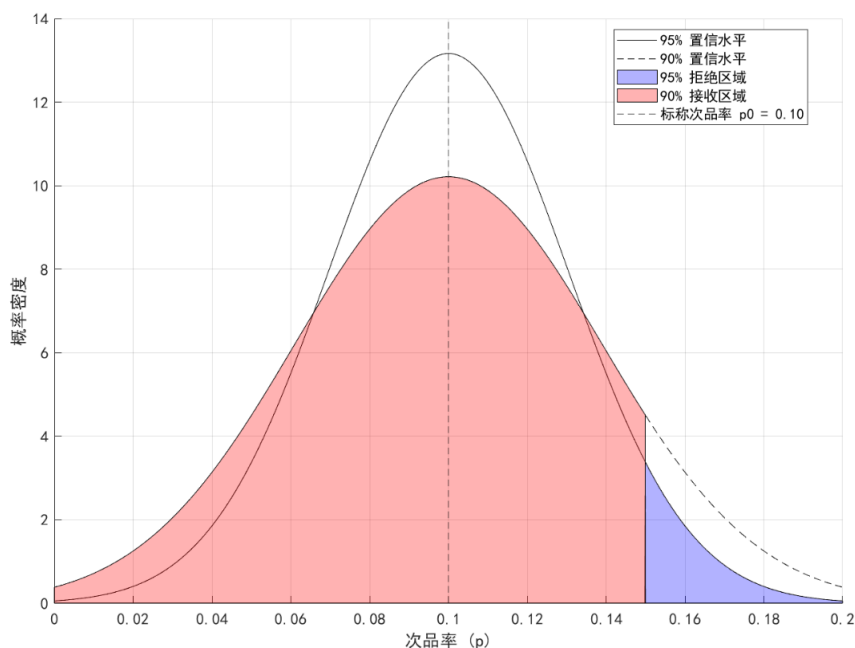


图 1 问题一结果可视化分析图

5.1.4 检验与分析

灵敏度分析^[6]主要目标是分析模型参数变化对检测结果的影响，对应到本题可知影响检测样本容量的关键参数为：供应商提供的标称值 p_0 、显著性水平 α 以及样本容量 n 。若 p_0 增加，则意味着供应商提供的产品质量预期下降，为确保检测效果，企业可能需要增加样本容量 n 以降低货品次品率过高的风险；为提高假设检验的保守性，降低显著性水平 α ，但会导致第I类错误的频发，因此需根据实际需求权衡各关键参数的调度，其中最直接影响检测成本及检测精度的因素为：样本容量 n 。本模型通过灵敏度分析可根据实际情况灵活调整参数以满足不同检测需求，找到满足企业不同情况下要求的最佳平衡点。

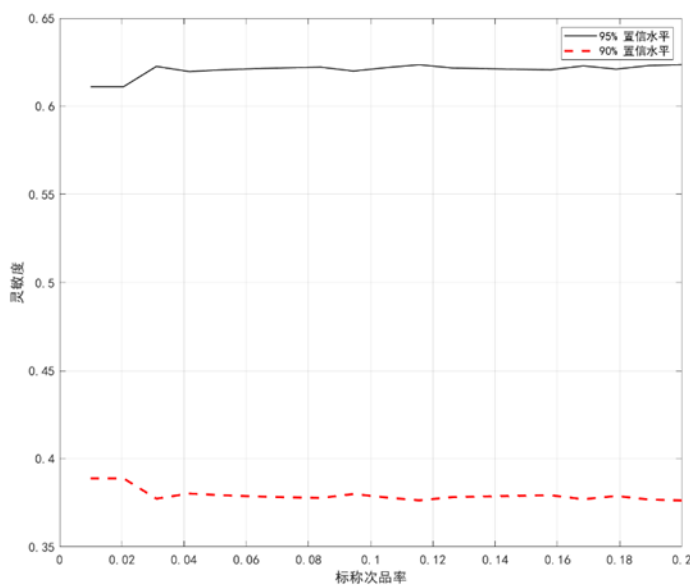


图 2 灵敏度分析

上图是对标称次品率的灵敏度分析。本文以0.1为初始标称次品率，以0.02为步长进行递增或递减，直至0.2和0.02结束。对上图分析得出，随着标称次品率的变化，90%

和95%置信水平的灵敏度变化幅度较小，几乎趋近于水平，说明本题建立的模型稳定性程度较高。

据题可知企业为尽可能的节约检测成本，需要设计出一个满足接收、拒收条件且尽可能小的检测样本容量，由于题中未给出供应商供应的单批次零配件总量，做95%和90%信度条件下的次品率于检验样品容量大小的线性关系图，当探测次品率与标称值相等时，取得最小样本容量值。

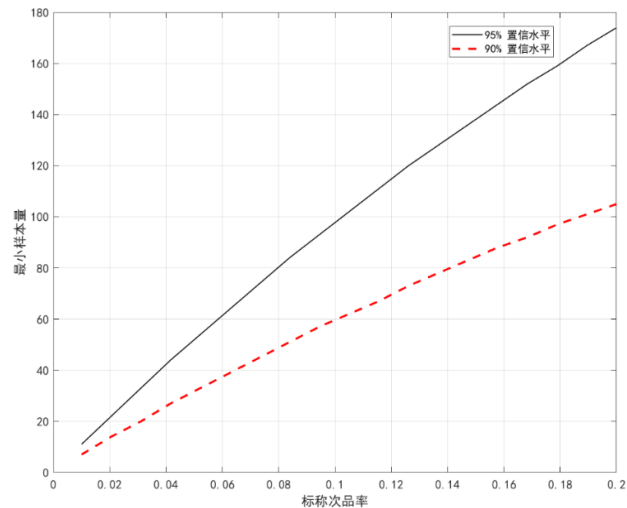


图 3 95%和 90%信度条件下的次品率与检验样品容量的灵敏度分析

上图是对95%和90%信度下的次品率与最小样本量的灵敏度分析。以0.1为初始标称次品率，以0.02为步长进行递增或递减，直至0.02和0.2截止。从图中可以看出，随着标称次品率的变化，最小样本量变化范围较小，说明最小样本量对标称次品率不灵敏。

5.1.5 小结

针对企业在采购畅销电子产品零配件时，为控制成本并保障产品质量，要求建立合理的模型对有效减少检测次数，缩小检测样本容量的问题，本文建立了基于分层抽样检测的二项分布与假设检验模型。首先，通过引入90%和95%的信度阈值，分别对应接收和拒收条件，构建了次品率与样本容量的关系模型。其次，利用二项分布的正态近似及中心极限定理，分析了次品数的分布情况，最后，通过计算不同信度阈值下次品率和检验样品容量的关系，确定了最小抽样量以满足企业检测需求，在控制检测成本、确保零配件质量的可信度的同时，为企业提供了科学、高效的进货检测方案。

5.2 问题二的模型建立与求解

5.2.1 模型建立的准备

本题要求对企业生产流程中的各个阶段做出合理决策，具体生产流程如下：

阶段1：零配件是否检测，若不检测，则零配件直接进入进入装配环节，否则，将检测出的零配件丢弃。

阶段2：成品是否检测，若不检测，则成品直接进入市场，否则，检测合格产品进入市场。

阶段3：不合格成品是否拆解，若不拆解，则不合格品丢弃，否则，将拆解的零配件重复步骤1和步骤2。

阶段4：企业无条件对不合格产品调换，对退回的不合格产品，重复步骤3。

针对上述的生产流程，结合问题背景中的零配件装配状况，将问题进行可视化转化，以便于简化分析。

下图展示该企业生产电子产品的流程的各个阶段的决策问题。结合问题背景，运用离散数学中逻辑推理^[7]的知识，初步分析零配件检测方案：首先，检测其中一个零配件，若该零配件不合格，满足产品不合格的充分条件，确定该成品一定不合格，不需要对另一零部件检测；若该零配件合格，需要进一步判断另一个零配件是否合格，确定产品是否符合合格的必要条件。接着，检测另一零部件，若该零部件合格，满足产品合格的必要条件，无法准确判断产品合格的情况，需要后续建立数学模型进行分析；若该零部件不合格，满足产品不合格的充分条件，说明该成品一定不合格，不需要进行后续判断。

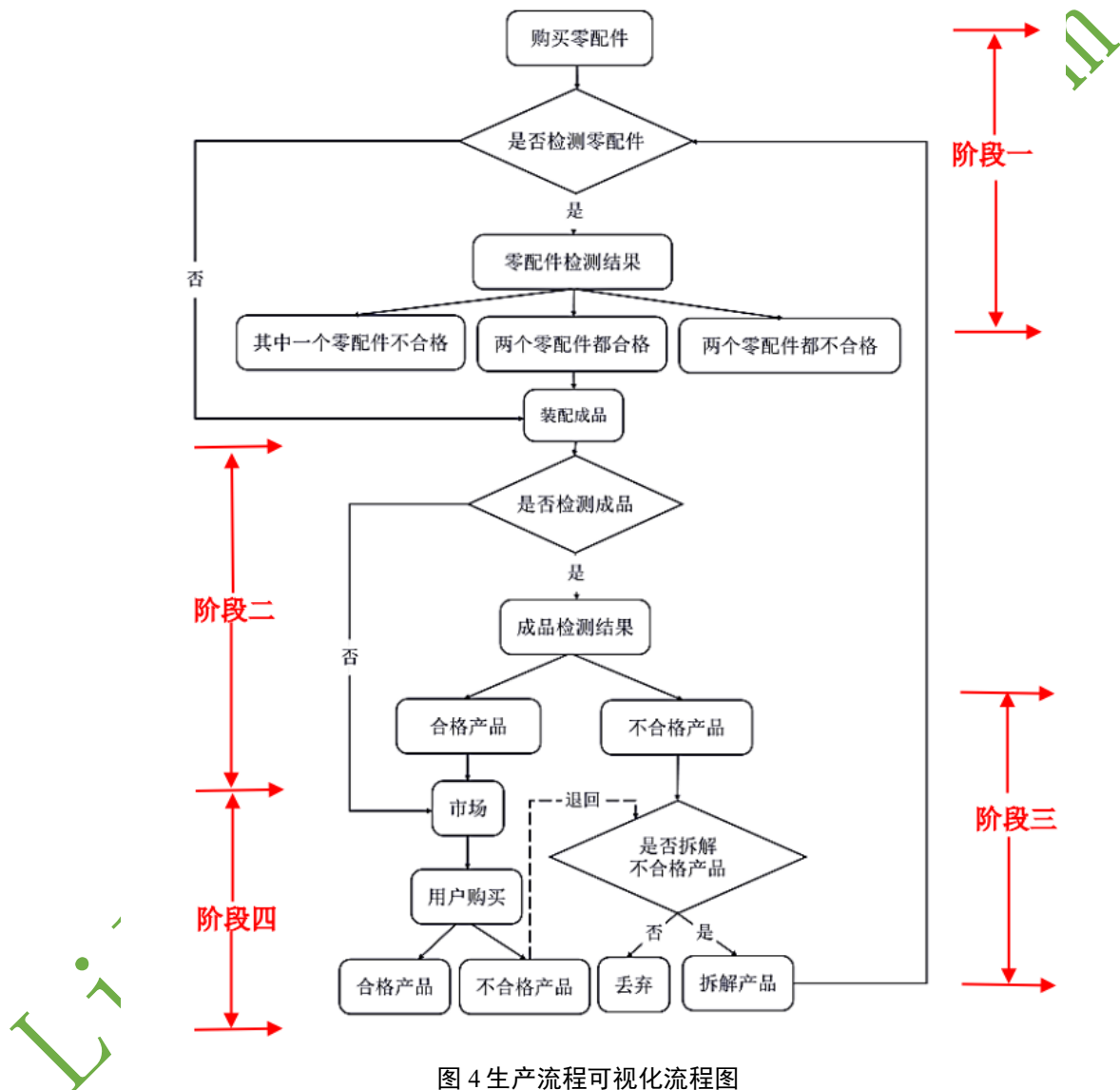


图 4 生产流程可视化流程图

5.2.2 模型的建立

5.2.2.1 多阶段决策模型

本文针对问题二中的各个阶段中的是否检测和拆解问题，建立0-1整数规划模型^[8]。最后，综合各个阶段规划问题和总耗费成本，以成本最小化作为目标函数，同时确定决策依据，建立出多阶段决策模型。

5.2.2.1.1 阶段一

阶段一是对零部件是否检测以及归属问题进行研究，运用0-1整数规划模型，对零部件检测进行决策判断。

决策变量为是否进行零配件1和零配件2检测，表示如下：

$$D_{1i} = \begin{cases} D_{11} = 1, \text{进行检测} \\ D_{10} = 0, \text{不进行检测} \end{cases}, i = 0, 1 \quad (19)$$

式中， D_{1i} 是零配件1的检测情况。

$$D_{2i} = \begin{cases} D_{21} = 1, \text{进行检测} \\ D_{20} = 0, \text{不进行检测} \end{cases}, i = 0, 1 \quad (20)$$

式中， D_{2i} 是零配件2的检测情况。

假设零部件在组装前的合格率为 ω_1 ，不合格率为 ω_2 ，购买零配件1和零配件2各有 n 个。零配件1和零配件2各有 n 个，零配件1的次品率为 q_1 ，零配件2的次品率为 q_2 ，成品次品率为 q_3 。

一个成品由一个零配件1和一个零配件2共同组成，对于各个零部件的检测进行的状态排列组合，共产生4种零部件检测情况，如下表所示：

表 2 检测状态分类

各个状态	检测状态
状态1	零部件1、2 都检测
状态2	只检测零部件1
状态3	只检测零部件2
状态4	零部件1、2都不检测

对于状态1，两种零部件均检测，并且即将组装成的成品，此时零部件1和零部件2在未组装之前是合格的。对于状态2，存在一种零部件接受检测，另一种零部件未接受检测，此时两个零部件在未组装前的合格概率，由未检测的零部件决定。又因为零部件1此时是合格的，其合格率为100%，则造成成品次品出现的情况是由零部件2决定的，此时零部件2的次品率与成品次品率相等。状态3与状态2的分析相同。对于状态4，两种零部件均未接受检测，需要根据两个零部件成品次品率比上假设的各个配件的次品率得到未检测前的次品率。各情况中未组装前的次品率，具体如下图所示：

表 3 各情况下零配件未检测前次品率

检测情况	两个零配件未组装前次品率	两个零配件未组装前合格品率
状态1	0	1
状态2	q_3	$1 - q_3$
状态3	q_3	$1 - q_3$
状态4	$1 - (1 - q_1)(1 - q_2)$	$(1 - q_1)(1 - q_2)$

此时，各个状态检测得出的次品为：

$$L_{1j} = \begin{cases} 0, j = 1 \\ n \cdot q_3, j = 2 \\ n \cdot q_3, j = 3 \\ n - n(1 - q_1)(1 - q_2), j = 4 \end{cases} \quad (21)$$

式中， j 表示各个检测状态， L_{1j} 为状态 j 的次品数。

各个状态的合格品数，表示为：

$$G_{1j} = \begin{cases} n, j=1 \\ n(1-q_3), j=2 \\ n(1-q_3), j=3 \\ n(1-q_1)(1-q_2), j=4 \end{cases} \quad (22)$$

式中， j 表示各个检测状态， G_{1j} 为状态 j 的合格品数。
各个状态产生的检测费用表示为：

$$W_1 = \sum_{i=1}^4 COST_1 D_{1i} n + COST_2 D_{2i} n \quad (23)$$

式中， W_1 表示每种状态产生的检测费用， $cost_1$ 表示零配件1的检测成本， $cost_2$ 表示零配件2的检测成本。

5.2.2.1.2阶段二

阶段二是对组装好的成品是否检测及其归属问题，运用0-1整数规划模型，对成品检测进行决策判断。

决策变量为是否对成品进行检测，表示为：

$$D_{pi} = \begin{cases} D_{p1} = 1, \text{ 进行检测} \\ D_{p2} = 0, \text{ 不进行检测} \end{cases}, i=1,2 \quad (24)$$

成品经过检测，意味着零部件全部为合格品：

$$p_1' = 1, p_2' = 0 \quad (25)$$

式中， p_1' 表示经过检测的零部件的合格率， p_2' 表示经过检测的零部件的不合格率。

成品未经过检测，意味着存在合格品和残次品，此时，零部件的合格率和不合格率表示如下：

$$p_1' = p_1, p_2' = p_2 \quad (26)$$

在问题一的基础上，分析各种情况的零部件组装后的成品次品率。假设零部件在组装后的合格率为 ζ_1 ，不合格率为 ζ_2 。对于状态1，两个零部件都合格的情况下，此时组装后的次品率与成品标称值相等；对于状态2和状态3，仅有1个零部件确定合格的情况下，组装后的次品率需要综合考虑产品本身的次品率和成品次品率；对于状态4，2个零部件均无法完全确定为合格品，组装后的次品率需要多次讨论每一种零部件和成品的次品率状况。基于容斥原理，计算成品未检测前，不合格概率，分析结果如下表所示：

表 4 零配件组装后次品率与合格率

检测情况	两个零配件组装后次品率	两个零配件组装后合格率
情况1	q_3	$1 - q_3$
情况2	$q_2 + (1 - q_2) \cdot q_3$	$(1 - q_1)(1 - q_3)$
情况3	$q_1 + (1 - q_1) \cdot q_3$	$(1 - q_2)(1 - q_3)$
情况4	$(q_1 q_3 - q_1 - q_2 + 1)(q_2 - 1) - 1$	$(1 - q_1)(1 - q_2)(1 - q_3)$

此时，组装后的成品次品数为：

$$L_{2j} = \begin{cases} G_{1j} \cdot q_3, j=1 \\ G_{1j} \cdot (q_2 + q_3 - q_2 q_3), j=2 \\ G_{1j} \cdot (q_1 + q_3 - q_1 q_3), j=3 \\ G_{1j} \cdot (q_1 q_3 - q_1 - q_2 + 1)(q_2 - 1) - G_{1j}, j=4 \end{cases} \quad (27)$$

式中, L_{2j} 为组装后状态 j 的成品的次品数, G_{1j} 为组装前零部件的合格品数。
同理, 组装后的成品合格品数为:

$$G_{2j} = \begin{cases} G_{1j} \cdot (1 - q_3), j = 1 \\ G_{1j} \cdot (1 - q_1)(1 - q_3), j = 2 \\ G_{1j} \cdot (1 - q_2)(1 - q_3), j = 3 \\ G_{1j} \cdot (1 - q_1)(1 - q_2)(1 - q_3), j = 4 \end{cases} \quad (28)$$

式中, G_{2j} 为组装后状态 j 的成品合格数。

假设经过阶段一零配件检测, 排除不合格的零配件, 得到的两类零部件各有 m 个, 成品有 S 个。根据阶段一可知, 零配件检测存在4种情况。依据上述四种情况, 分别确定各自所剩零部件数量。

对于状态1, 两个零部件均经过检测, 过滤掉残次品, 所剩可以组装为成品的零配件数量表示为:

$$m_1 = \min \{n(1 - q_1), n(1 - q_2)\} \quad (29)$$

对于状态2, 存在一个零部件经过检测, 过滤掉残次品, 所剩可以组装为成品的零配件数量表示为:

$$m_2 = \min \{n \cdot (1 - q_1), n\} \quad (30)$$

对于状态3, 分析流程与情况2类似, 所剩可以组装为成品的零配件数量表示为:

$$m_3 = \min \{n \cdot (1 - q_2), n\} \quad (31)$$

对于状态4, 两个零配件均为经过检测, 直接组装为成品, 此时, 组装为成品的零配件数量表示为:

$$m_4 = n \quad (32)$$

由此, 当成品未检测时, 可得出经过阶段2的成品有 S 个, 次品有 $(m_j - S)$ 个:

$$S = m_j \cdot p_1, j = 1, 2, 3, 4 \quad (33)$$

$$m_j - S = m_j \cdot p_2, j = 1, 2, 3, 4 \quad (34)$$

此时, 各个情况产生的检测费用表示为:

$$W_2 = D_{pi} \cdot COST_3 \cdot m_j \quad (35)$$

式中, $COST_3$ 为成品检测成本

当成品不进行检测时, 产生的损失为:

$$F = \sum_{i=1}^r m_j \cdot \kappa \cdot \eta_i \cdot \chi_i, i = 1, 2 \quad (36)$$

式中, m_j 是成品数量, κ 是未检测成品概率, η 是零部件调换产生的费用, χ 是零部件的拆解费用。

5.2.2.1.3阶段三

阶段三是对不合格产品是否拆解问题的研究, 运用0-1整数规划模型, 对不合格产品拆解进行决策。

决策变量为是否对不合格成品进行拆解, 表示为:

$$D_{ci} = \begin{cases} D_{c1} = 1, \text{进行拆解} \\ D_{c2} = 0, \text{不进行拆解} \end{cases} \quad (37)$$

由阶段二可知, 当成品未检测直接进入市场, 用户可能购买到的不合格品的数量为 $m_j - s$, 此时不合格成品拆解产生的拆解成本为:

$$W_{31} = D_{ci} \cdot COST_4 \cdot (m_j - S) \quad (38)$$

式中， $COST_4$ 为成品拆解费用， W_{31} 为阶段三中成品未检测进入市场产生的拆解成本。

对不合格成品不进行拆解时，此时拆解产生的成本为0元。

当检测后的不合格成品拆解时，此时产生的拆解成本，包括成品中的次品率和超出配对的零件数，表示为：

$$W_{32} = D_{ci} \cdot COST_4 \cdot (m_j - S + S \cdot p_3) \quad (39)$$

式中， W_{32} 为阶段三中成品检测后进入市场产生的拆解成本， p_3 为成品的次品率。

本阶段的不合格产品是否拆解问题，结合上述两阶段，确定共有16种方案。运用二叉树的相关知识，列举出16种具体方案。

由此，得出16种方案的具体情况，为了清晰表达，本文运用表格形式在此展示：

表 5 拆件方案展示

方案	是否检测配件1	是否检测配件2	是否检测成品	是否拆件不合格品
1	T	T	T	T
2	T	T	T	F
3	T	T	F	T
4	T	T	F	F
5	T	F	T	T
6	T	F	T	F
7	T	F	F	T
8	T	F	F	F
9	F	T	T	T
10	F	T	T	F
11	F	T	F	T
12	F	T	F	F
13	F	F	T	T
14	F	F	T	F
15	F	F	F	T
16	F	F	F	F

5.2.2.1.4阶段四

在阶段四中，主要存在企业对不合格产品进行调换，产生调换损失。结合上述三个阶段的模型构建流程，通过对成品是否检测的讨论，分析调换损失产生的成本。由于，阶段三已经将不合格品进行拆解或者丢弃，则经过成品检测得到不合格品已不存在。本题只需考虑成品未检测，直接进入市场后，产生的调换损失。

当成品未检测直接进入市场，用户可能购买到的不合格品的数量为 $m_j - s$ ，此时不合格成品的调换损失为：

$$W_4 = COST_5 \cdot (m_j - s) \quad (40)$$

式中， W_4 为未检测直接进入市场的不合格品的调换损失成本， $COST_5$ 为不合格品的调换损失。

综上，本题的总成本函数为：

$$W = W_1 + W_2 + W_{3i} + W_4 + R, i = 1, 2 \quad (41)$$

$$W = n \cdot (COST_1 \cdot D_{1i} + COST_2 \cdot D_{2i}) + m \cdot COST_3 \cdot D_{pi} + (m - s) \cdot (COST_4 D_{ci} + COST_5 + R) \quad (42)$$

因此，本题的目标函数最小化总成本可以表示为：

$$\min W \quad (43)$$

5.2.3 模型的求解

5.2.3.1 模型求解步骤

本题运用穷举算法^[9]列举所有可能的决策情况，寻找到最优解。具体步骤如下所示：

1. 穷举策略：代码首先定义了一组策略，包括是否检测零配件1、是否检测零配件2、是否检测成品以及是否拆解不合格成品。然后通过嵌套循环遍历这些策略的所有可能组合，生成了一个包含所有策略的数组`strategies`。

2. 穷举数量范围：代码定义了一个数量范围 `n_range`，从100到1000，步长为100。这意味着将考虑每个情况下不同数量的产品(例如100件、200件等)。

3. 穷举情况：代码定义了一组情况`cases`，每个情况包含了零配件和成品的次品率、检测成本、装配成本、市场售价、替换损失和拆解成本等信息。然后通过外层循环遍历这些情况。

4. 计算总成本和次品率：对于每个情况和策略组合，代码调用`calc_total_cost`函数来计算总成本和次品率。这个函数根据给定的策略和数量计算了零配件和成品的检测成本、损失成本、拆解成本，并最终计算出总成本和次品率。

5. 存储结果：每次计算后，代码将结果存储到一个表格`results`中，包括情况编号、数量、总成本、次品率以及其他相关信息。

6. 输出结果：最后，代码将结果表格转换为一个结构化数组`results_table`，并将其保存到Excel文件中。总结起来，这段代码通过穷举所有可能的策略组合和数量范围，计算了每种情况下的总成本和次品率，并将结果整理成表格形式以便进一步分析和决策。

本题通过穷举法对多阶段决策模型进行求解,求解的具体流程产生的结果见附件 `t2_results.xlsx`。本题在已求得的决策方案中选取其中最优方案。比较分析同一零部件数量中的决策方案，选定花费最少的作为最优方案。本文仅从零配件数量为100选择出了最优决策方案及其指标结果，如下图所示：

表 6 最优决策方案

情况	零配件数量	方案	是否检测零部件1	是否检测零部件2	是否检测成品	是否拆解不合格品	总成本	次品率
1	100	16	否	否	否	否	2380	10%
2	100	16	否	否	否	否	2560	20%
3	100	13	否	否	是	否	2620	5%
4	100	2	是	是	是	否	2600	10%
5	100	12	否	是	否	否	2460	10%
6	100	16	否	否	否	否	2310	5%

5.2.3.2 决策依据分析

由于本题并未给出具体的决策方向，因此，本文运用反推法确定决策依据。通过控制变量法，结合确定各个阶段的决策依据。考虑到零配件1和零配件2是否检测的决策反映的本质相同，因此，本题仅确定其中一个零部件检测状态变化，其他决策因素不变。运用Origin绘图，将其变化情况可视化表达。

当零配件个数为500时，此时，仅改变零配件1检测状态，其他因素不变，可视化分析图如下：

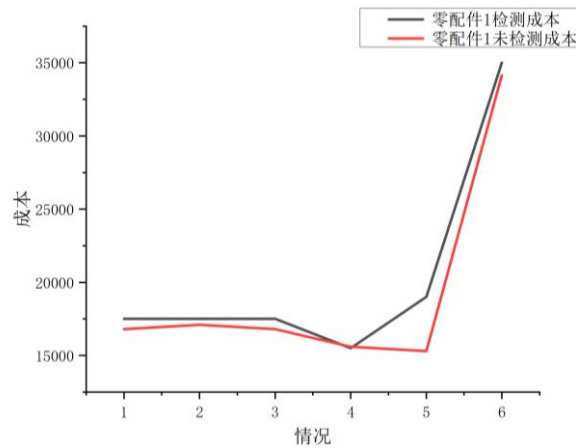


图 5 不同零配件检测状态成本对比图

由上图可以看出，零配件1的检测成本在各个情况下，始终大于零配件1的未检测成本。

当零部件不进行检测时，产生的损失为：

$$T = \sum_{\sigma=1}^f n \cdot \gamma_i \cdot \eta_i \cdot \chi_i, i=1,2 \quad (44)$$

式中， n 为每个零部件的总个数， $f=n \cdot \gamma$ 表示未接受检测的零部件的数量， γ 为未检测的零部件的概率， η 是零部件调换产生的费用， χ 是零部件的拆解费用。

判断零部件是否需要检测，主要从检测成本和不检测零部件产生损失的角度出发。若检测零部件的成本大于不检测零部件产生的损失，则不检测零部件；若检测零部件的成本小于不检测零部件产生的损失，则检测零部件。因此，本题对于检测零部件的决策依据是不检测零部件产生的损失期望小于等于检测零部件产生的成本，即

$$E(T) \leq COST_j \quad (45)$$

式中， $COST_j$ 表示检测零部件 j 产生的成本。

$$\text{其中, } E(T) = \frac{1}{f} \sum_{\sigma=1}^f n \cdot \gamma_i \cdot \eta_i \cdot \chi_i, i=1,2。$$

同理，当零配件个数为500时，仅改变成品检测的状态，其他因素不变，可视化分析图如下：

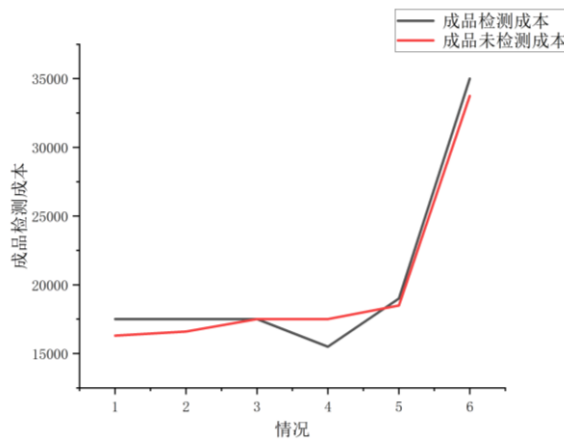


图 6 成品检测状态成本对比分析图

从上图中可以看出，是否检测成品在80%的情况下，成品检测成本始终大于未检测成品成本(此决策标准对情况四始终不适用)。判断成品是否需要检测，主要从成本角

度出发。本题对于成品检测的决策依据是不检测成品产生的损失期望小于等于检测成品产生的成本，即：

$$E(F) \leq COST_3 \quad (46)$$

式中， $COST_3$ 为单个成品的检测成本。

$$\text{其中， } E(F) = \frac{1}{r} \sum_{\psi=1}^r m_j \cdot \kappa \cdot \eta_i \cdot \chi_i, i=1,2。$$

同理，在零配件个数为500个时，不合格品拆解状态改变，其他因素不变，可视化分析图如下：

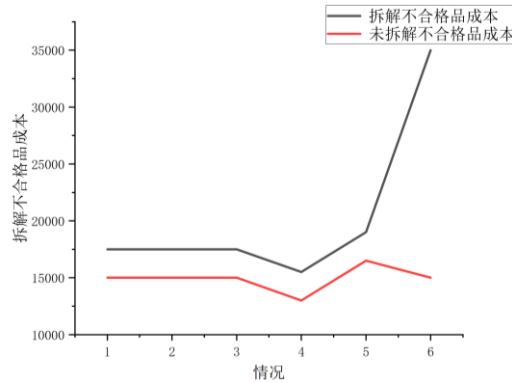


图 7 不合格品拆解状态成本对比图

从上图可以分析得出，在各个情况下，拆解不合格品的成本始终大于未拆解的不合格品成本。因此，对不合格品是否拆解的决策依据是未拆解不合格品的损失小于拆解不合格品的成本，表示为：

$$E(J) \geq COST_4 \quad (47)$$

式中， $COST_4$ 为单个不合格品拆解成本， $E(J)$ 表示未拆解的不合格品的损失。

$$\text{其中， } E(J) = \frac{1}{\delta} \sum_{\tau=1}^{\delta} (m-s) \cdot (c_i + v_i)，\text{此时， } c_i \text{表示第 } i \text{ 种情况的装配成本， } v_i \text{表示第 } i$$

种情况的检测成本， δ 表示不合格品数量，即 $\delta=m-s$ 。

综上，情况4的决策依据为：不检测零部件产生的损失期望小于等于检测零部件产生的成本、未拆解不合格品的损失小于拆解不合格品的成本；情况1/2/3/5/6的决策依据为：不检测零部件产生的损失期望小于等于检测零部件产生的成本、未拆解不合格品的损失小于拆解不合格品的成本和不检测成品产生的损失期望小于等于检测成品产生的成本。具体指标结果见附录test2_results.xlsx。

5.2.4 检验与分析

5.2.4.1 蒙特卡洛检验

蒙特卡洛法^[10]也称统计模拟法，是把概率现象作为研究对象的数值模拟方法，是按照抽样调查法求取统计值推定未知特性量的计算方法。

蒙特卡洛法的基本思想是：若已知随机变量 $X_1, X_2, \dots, X_K$ 的概率分布及极限状态方程 $Z=g(X_1, X_2, \dots, X_K)$ ，根据各随机变量 $X_1, X_2, \dots, X_K$ 分布利用蒙特卡洛算法产生相应分布的随机数 $x_1, x_2, \dots, x_k$ ，代入极限状态方程，得：

$$z_1 = g(x_1, x_2, \dots, x_k) \quad (48)$$

作 K 次这样的试验，可得随机变量 K 的一组样本： $k_1, k_2, \dots, k_n$ ，若其中有 M 个 $z < 0$ ，当

$N \rightarrow \infty$ 时, 根据伯努利大数定理

$$P_j = P\{g(X_1, X_2, \dots, X_k) \leq 0\} = M / N \quad (49)$$

对这 N 个 $k_1, k_2, \dots, k_n$ 求其均值 μ_j 和均方差 σ_j , 从而可计算可靠度指标: $\beta = \mu_j / \sigma_j$ 。

针对于此, 本文在`calc_total_cost`函数中添加随机性, 模拟次品率的不确定性。接着, 运用蒙特卡洛模拟估计次品率的值, 对比添加随机后的次品率变化情况。

由于蒙特卡洛算法能够使用的前提是样本量符合正态分布, 因此, 本文假设次品率服从正态分布。为了进一步检验模型的准确性, 使用`normrnd`函数生成随机数表示次品率。以下是蒙特卡洛算法的具体步骤:

1. 产生 $[0,1]$ 均匀分布的随机次品率;
2. 确定次品率的正态分布的参数为均值和标准差
3. 分别对每个次品率进行 1000 次重复抽样, 得到各次品率的占比离散值。
4. 对数据进行分组处理, 计算出每个次品率发生的概率, 绘制概率分布函数图。

通过上述算法求解, 得出为添加随机性的次品分布和添加随机性的次品分布概率及其对比分析图:

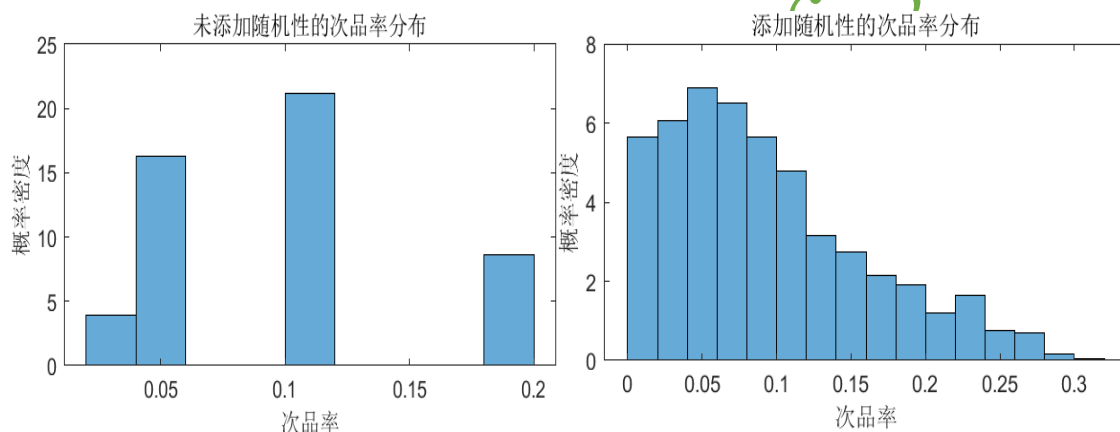


图 8 添加随机性次品率(左)和未添加随机性次品率(右)分布图

通过对上述添加随机性次品率和未添加随机性次品率分布的对比分析, 得出以下结论:

1. 与未添加随机性的次品率相比, 添加随机性的次品率在 $0-0.3$ 内均有分布, 覆盖范围较大, 条带变宽, 表明零部件的次品率对成品的次品率影响较大。
2. 与未添加随机性的次品率相比, 添加随机性的次品率的占比分布近似符合正态分布, 表明次品率分布更加贴合实际, 为真实情况下提供一系列依据。
3. 规划效果较好。与未添加随机性次品率相比添加随机性次品率的不同情况下, 次品率变化浮动较小, 因此, 说明本题规划效果较好。

5.2.5 小结

首先, 将本题的决策要求运用了流程图的形式将其可视化表达, 便于后续分析。接着, 运用 $0-1$ 整数规划模型和概率论相关知识, 计算出各个决策阶段的零配件或成品的次品率和合格率, 进而表示出了各个阶段的花费成本并将其汇总, 以最小化成本为目标函数建立多阶段决策模型。然后, 运用穷举法对模型进行求解, 确定了各个情况下的最优方案。最后, 运用蒙特卡洛算法对模型进行检验, 比较分析添加随机性次品率和未添加随机性次品率的状况, 结果表明本题模型建立较好, 更加契合实际。

5.3 问题三的建立与求解

本题与第二问决策方式类似，在问题二的多工序多阶段决策模型基础上增设多个非成品的阶段性组装过程，最后对成品进行组装，据此多类型阶段的组装工序，给出每阶段的决策依据及相应指标。

5.3.1模型建立的准备

首先，将问题二中的多工序多阶段决策模型结构化为一个子工序，在此基础上增添不同组装情况下的零配件规格(个数、次品率、购买单价、检测成本)。接着，将半成品的规格(个数、次品率、购买单价、检测成本、装配成本、拆解费用)反代入子工序中，得出最终成品在不同检测情况下的不同规格。最后，对最终成品的多种组装情况进行关于投放市场的决策分析，将市场销售情况与调换损失情况计入直接影响指标，对具体生产情况进行分析。

5.3.2模型的建立

5.3.2.1多工序多阶段决策模型

5.3.2.1.1子工序预测模型

据题中图1的两道工序、8个零配件的组装情况，先对8个零配件是否检测进行分析，由于第二问已给出了两个零配件在组装成品时的子工序，此模型首先考虑三个零配件在组装成品时的子工序。

是否检测零配件的决策变量：

$$D_{ji} = \begin{cases} D_{j1} = 1, \text{进行零配件检测} \\ D_{j0} = 0, \text{不进行零配件检测} \end{cases}, j=1,0 \quad (50)$$

式中， j 为零配件的类型编号，据题意可知，零配件共有 $x=8$ 种。

记零配件 j 的次品率为 q_j ， $COST_j$ 为零配件 j 的检测成本，由于此处先考虑只有三个零配件时组装半成品的子工序，在考虑了是否检测的决策变量后，在未组装成半成品之前，记三个零配件组的合格的概率为 P_1 ，不合格的概率为 P_2 ，为计算 P_1 、 P_2 ，可分为下述4种情况：

表 7 各个情况下的零配件合格率与次品率

情况	零配件1 检测情况	零配件2 检测情况	零配件3检 测情况	P_1	P_2
1	$D_{1i} = 1$	$D_{2i} = 1$	$D_{3i} = 1$	100%	0%
2	$D_{1i} = 0$	$D_{2i} = 1$	$D_{3i} = 1$	$1 - q_1$	q_1
3	$D_{1i} = 0$	$D_{2i} = 0$	$D_{3i} = 1$	$(1 - q_1)(1 - q_2)$	$q_1(1 - q_2) + q_2(1 - q_1) + q_1q_2$
4	$D_{1i} = 0$	$D_{2i} = 0$	$D_{3i} = 0$	$(1 - q_1)(1 - q_2)(1 - q_3)$	$q_1(1 - q_2)(1 - q_3) + q_2(1 - q_1)(1 - q_3) + q_1q_2(1 - q_3)$
5	$D_{1i} = 1$	$D_{2i} = 0$	$D_{3i} = 1$	$1 - q_2$	q_2
6	$D_{1i} = 1$	$D_{2i} = 1$	$D_{3i} = 0$	$1 - q_3$	q_3
7	$D_{1i} = 1$	$D_{2i} = 0$	$D_{3i} = 0$	$(1 - q_2)(1 - q_3)$	$q_2(1 - q_3) + q_3(1 - q_2) + q_2q_3$
8	$D_{1i} = 0$	$D_{2i} = 1$	$D_{3i} = 0$	$(1 - q_1)(1 - q_3)$	$q_1(1 - q_3) + q_3(1 - q_1) + q_1q_3$

若采取不同的关于三个零配件决策变量，则会产生相应的检测费用：

$$W_{tk(k=1,2,3)} = COST_1 \cdot D_{1i} \cdot n_1 + COST_2 \cdot D_{2i} \cdot n_1 + COST_3 \cdot D_{3i} \cdot n_1 \quad (51)$$

式中， $COST_1$ 为零配件1的检测成本， $COST_2$ 为零配件2的检测成本， $COST_3$ 为零配件3的检测成本； t 为计数值，主要用于下文中对前期投入成本的计算，此处 $t=1$ 。

是否检验半成品的决策变量：

$$D_{Pi} = \begin{cases} D_{P1} = 1, \text{进行半成品检测} \\ D_{P0} = 0, \text{不进行半成品检测} \end{cases}, i = 0, 1 \quad (52)$$

由于第一道对零配件的检测工序会丢弃检测出的次品，原零配件数量发生改变，在考虑是否检测半成品时，应首先考虑现有的零配件余量 m_1 ：

$$m_1 = n_1 \cdot P_1 \quad (53)$$

式中 n_1 为原零配件数目， P_1 为不同情况下的零配件组的合格率。

记 ε_1 为上述零配件组在装配后的合格率， ε_2 为上述零配件组在装配后的次品率，表达式如下：

$$\begin{cases} \varepsilon_1 = (1 - \varepsilon)P_1 \\ \varepsilon_2 = P_2 + \varepsilon P_1 \end{cases} \quad (54)$$

式中， ε 为表二所给的半成品1的次品率， $\varepsilon=10\%$ 。

由此可知若剔除检测结果为次品的半成品，余下的半成品数目为：

$$S = m_1 \cdot \varepsilon_1 \quad (55)$$

既已知半成品的次品率与余量，根据前期对零配件组的检测费用和半成品拆解后的利用率，对不合格半成品是否进行拆解进行分析，是否拆解半成品的决策变量如下：

$$D_{Ci} = \begin{cases} D_{C1} = 1, \text{进行半成品拆解} \\ D_{C0} = 0, \text{不进行半成品拆解} \end{cases}, i = 1, 0 \quad (56)$$

首先，先对组装半成品前期过程花费的检测成本进行计算：

$$W_{tk} = D_{Pi} \cdot \text{COST}_4 \cdot m_1 \quad (57)$$

式中， COST_4 为单个半成品的检测费用， $t=2$ 。

若选择对符合条件的半成品进行拆解，则需花费的拆解费用如下：

$$W_{rk} = D_{Ci} \cdot \text{COST}_5 \cdot (m_1 - S_1) \quad (58)$$

式中， COST_5 为单个半成品的拆解费用， r 为计数值，代表企业在生产制造时的第 r 笔拆解费用，此处 $r=1$ 。

根据不同零配件组下的半成品，分别计算其相应的前期投入成本：

$$W_0 = \sum_{k=1}^3 (\sum_{t=1}^t W_{tk} + \sum_{r=1}^r W_{rk}) \quad (59)$$

半成品1与半成品2都是由三个零配件组装而成，其在组装为产品前的前期投入成本为：

$$W_1 = W_2 = \sum_{k=1}^3 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}), r = 1 \quad (60)$$

式中， W_1 、 W_2 分别为半成品1、2的前期投入成本。

由于用于组装半成品1、2的零配件次品率相同，易知半成品1、2的合格率相同，即：

$$\begin{cases} \varepsilon_1 = \varepsilon_3 \\ \varepsilon_2 = \varepsilon_4 \end{cases} \quad (61)$$

式中 ε_3 、 ε_4 分别为半成品2的合格率和不合格率。

半成品3是由两个零配件组装而成，其在组装为产品前的前期投入成本 W_3 为：

$$W_3 = \sum_{k=1}^2 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}), r = 1 \quad (62)$$

又因半成品的组装工序与第二问相同，可直接调用问题二中的表达式，直接表示

半成品3的合格率 ε_5 :

$$\varepsilon_5 = (1 - \varepsilon)P_5 \quad (63)$$

在已知半成品组的各半成品前期投资成本和相应的合格率的情况下, 考虑到半成品并未加工为成品, 不具备进入销售市场的基础条件, 所以此处不考虑购买零配件的成本问题, 只对可能出现的拆解半成品情况进行投入成本分析。在考虑了半成品的拆解问题后, 会引出是否对拆解所得的零配件进行第二轮检验, 和组装问题, 此处先忽略这一附加问题, 只对第一轮产品生产工序进行分析, 由此可得, 下面本模型应先分析半成品组组装后是否检测最终产品的决策变量:

$$D_{ji} = \begin{cases} D_{j1} = 1, \text{进行半成品检测} \\ D_{j0} = 0, \text{不进行半成品检测} \end{cases}, i = 0, 1 \quad (64)$$

分情况计算不同半成品组的合格率:

表 8 不同半成品组的合格率及次品率

情况	半成品1的 检测情况	半成品2的 检测情况	半成品3的 检测情况	合格率 Q_1	次品率 Q_2
1	$D_{1i} = 1$	$D_{2i} = 1$	$D_{3i} = 1$	100%	0%
2	$D_{1i} = 0$	$D_{2i} = 1$	$D_{3i} = 1$	$1 - \varepsilon_2$	ε_2
3	$D_{1i} = 0$	$D_{2i} = 0$	$D_{3i} = 1$	$(1 - \varepsilon_2)(1 - \varepsilon_4)$	$\varepsilon_2\varepsilon_3 + \varepsilon_4\varepsilon_1 + \varepsilon_2\varepsilon_4$
4	$D_{1i} = 0$	$D_{2i} = 0$	$D_{3i} = 0$	$\varepsilon_1\varepsilon_3\varepsilon_5$	$1 - \varepsilon_1\varepsilon_3\varepsilon_5$
5	$D_{1i} = 1$	$D_{2i} = 0$	$D_{3i} = 1$	ε_3	ε_4
6	$D_{1i} = 1$	$D_{2i} = 1$	$D_{3i} = 0$	ε_5	ε_6
7	$D_{1i} = 1$	$D_{2i} = 0$	$D_{3i} = 0$	$\varepsilon_3\varepsilon_5$	$\varepsilon_4\varepsilon_5 + \varepsilon_6\varepsilon_4 + \varepsilon_4\varepsilon_6$
8	$D_{1i} = 0$	$D_{2i} = 1$	$D_{3i} = 0$	$\varepsilon_1\varepsilon_5$	$\varepsilon_2\varepsilon_5 + \varepsilon_2\varepsilon_5 + \varepsilon_2\varepsilon_6$

由上表易知, 在进行检验之后可得成品的合格率为:

$$\begin{cases} R_1 = (1 - \rho)Q_1 \\ R_2 = Q_2 + \rho Q_1 \end{cases} \quad (65)$$

易知若剔除检测结果为次品的成品, 余下的成品数目 g 为:

$$g = S \cdot Q_1 \quad (66)$$

基于成品的合格率, 将商品投放入市场, 对有一定机率出现的调换情况进行分析:

$$D_{Ui(i=0,1)} = \begin{cases} D_{U1} = 1, \text{进行不合格成品调换} \\ D_{U0} = 0, \text{不进行不合格成品调换?} \end{cases} \quad (67)$$

其是否调换的根本决策变量为: 所得利润大于投入成本, 下面先对单个成品的具体利润进行分析:

$$\begin{cases} M' \geq W' \\ W' = \sum_{j=1}^8 x_j + \sum_{i=1}^3 \delta_i + 2 \sum_{k=1}^3 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}) + \sum_{k=1}^2 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}) - h \cdot b_0, r = 1 \end{cases} \quad (68)$$

式中, M' 为成品的市场售价, W' 为单个成品的投入成本, h 为单次调换产生的损失, b_0 为成品在投入市场后具体调换次数, x_j 为零配件 j 的购买单价, δ_i 为半成品 i 的装配成本, 接下来对企业生产这批商品的总利润进行分析:

$$\begin{cases} M \geq W \\ W = n_0 \cdot \sum_{j=1}^8 x_j + S \cdot \sum_{i=1}^3 \delta_i + (S_1 + S_2) \cdot W_0 + S_3 \cdot W_3 - h \cdot b, \quad b \leq 3 \\ S_1 + S_2 + S_3 = S \end{cases} \quad (69)$$

式中， M 为所有成品的市场总售价， W 为企业为生产这批成品的总投入成本， h 为单次调换产生的损失， b 为所有成品在投入市场后全部调换次数， S_1 、 S_2 、 S_3 分别为零配件1、2、3的总量。

若满足目标函数：

$$\max F = M - W \quad (70)$$

则同意对客户进行无偿调换处理。

下面对半成品的拆解问题进行具体分析，由于上述表达式(56)已给出了是否拆解半成品的决策变量，此处本模型只需分析拆解半成品对企业生产情况的影响，当半成品未检测直接进入组装工序，企业可能生产的到的不合格成品的数量为 $S \cdot g$ ，此时不合格半成品拆解产生的拆解成本为：

$$W_{rk} = D_{Ci} \cdot COST_5 \cdot (m - S), r = 1, 2, 3, \dots \quad (71)$$

式中， $COST_5$ 为半成品拆解费用， W_{rk} 为半成品未检测进入市场前产生的拆解成本。对不合格半成品不进行拆解时，此时拆解产生的成本为0元。

当拆解检测后的不合格半成品时，此时产生的拆解成本，包括半成品中的次品率和超出配对的零件数，表示为：

$$W_{rk} = D_{Ci} \cdot COST_5 \cdot (S_j - g + g \cdot Q_2), r = 1, 2, 3, \dots \quad (72)$$

式中， W_{rk} 为半成品检测后进入市场产生的拆解成本， Q_2 为半成品的次品率。

对于不合格半成品是否拆解的问题，结合上述对半成品在检测前后的拆解费用分析，易知当有8个零配件时，每个零配件都可以检测或不检测，共有 2^8 种组合；每个半成品也可以检测或不检测，每个子工序有3个半成品，共 2^3 种组合；最终对成品有两种拆解情况：检测后发现为次品拆解、调换后拆解，再加上是否检测的两种情况共4种拆解情况。最终，确定共有 2^{13} 种策略组合。运用二叉树的相关知识，列举出8192种具体拆解方案，并对此进行分析。

5.3.3模型的求解

5.3.3.1求解具体步骤

本模型主要运用穷举算法列举所有可能的决策情况，寻找企业生产成品的最优决策组合。具体步骤如下：

步骤1：初始化组件、半成品和成品的参数。首先，定义组件、半成品和成品的参数，包括次品率、采购价格、检测成本、装配成本、拆解成本等对销售过程的影响因素，便于用于后续对每种策略下的利润计算。

步骤2：生成所有可能的策略组合，使用二进制表示法生成所有可能的检测策略组合，对每个半成品组在考虑拆解情况后的所有情况进行拆分组合分析。

步骤3：初始化储存空间，用于放入每个每个策略的结果，并为每个策略可能产生的结果进行具体规范(零配件、半成品、成品个数，拆解、检测总次数)。

步骤4：代码遍历所有企业可能在生产过程中调用的组合策略，调用 *calculate expected cost* 函数计算出每个策略的总成本和次品率并生成对当前策略的描述。最后使用 *excel* 列表存储具体组合策略下的所有投入成本和利润。

步骤5：调用函数 *generate_strategy_description* 生成每个策略下关于零配件、半成品、成品的检测抉择因素，半成品、成品的 r 轮拆解抉择因素的描述字符串，用于记录每个

策略的具体操作。

步骤6: 函数返回预期的总成本和次品率，用于对所有策略组合的结果比较，选取出一最适宜企业生产电子产品的策略。

通过执行上述算法具体步骤步骤，所得全部决策方案(见附件t3_results.xlsx)，在此基础上根据最小化成本作为目标，确定最优决策方案为：不检测零配件1，不检测零配件2，不检测零配件3，不检测零配件4，不检测零配件5，不检测零配件6，不检测零配件7，不检测零配件8，不检测半成品1，不检测半成品2，不检测半成品3，不检测成品，不拆解不合格成品。

5.3.3.2决策依据

第三问在零配件组装、检验和成品检验的工序上决策近似于第二问，此处不多加赘述，但由于本题并未给出具体的决策方向，因此，本文选用反推法确定决策依据。通过控制变量法，结合确定各个子工序的决策依据。考虑到半成品1和半成品2是否检测的决策反映的本质相同，因此，本题仅确定其中一个半成品检测状态变化，其他决策因素不变。

当零配件个数区间范围为1~1000时，此时，只有零配件组检测状态改变，其他因素不变，可视化分析图如下：

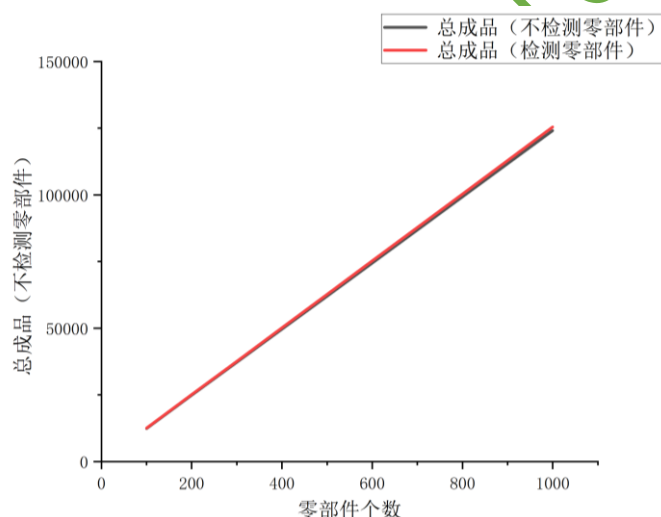


图9 是否检测零部件成本对比图

判断零部件是否需要检测，主要从成本和损失角度出发。若检测零部件后的总成本大于不检测零部件产生的总成本，则不检测零部件，反之，则检测零部件，据图可知，零配件是否检测对总成本的影响曲线近乎重叠，即零配件的检测对企业生产过程中的总成本无较大影响，若企业选择不检测零配件对最终生产成本的节约可近似不计。

当零配件个数区间范围为1~1000时，此时，只有半成品组检测状态改变，其他因素不变，可视化分析图如下：

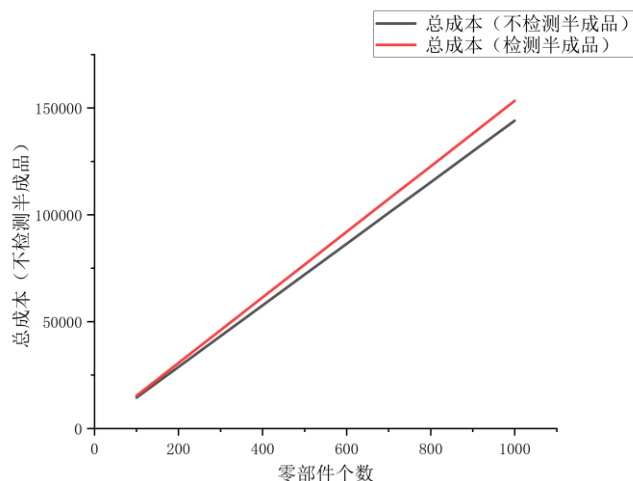


图 10 是否检测半成品成本对比图

由上图中易知，若企业选择在生产过程中对半成品进行检测，生产总成本在各个情况下，始终大于半成品组未检测时的成本。要判断半成品组是否需要检测，主要从成本角度出发，本题对于半成品组检测的决策依据是不检测半成品组产生的损失期望小于等于检测半成品组产生的成本，据上述模型的建立中分析的半成品在组装前的前期投入成本，可知选择不检测半成品组可有效缩减生产总成本。

当零配件个数区间范围为1~1000时，此时，只有半成品组检测状态改变，其他因素不变，可视化分析图如下：

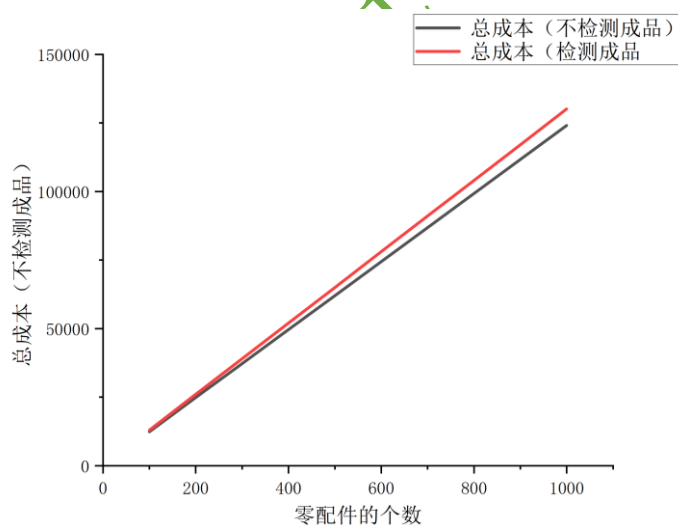


图 11 是否检测成品成本对比图

由上图可知，若企业选择对成品进行检测，生产总成本在各个情况下，始终大于成品的未检测成本。

下面对半成品是否选择拆解的工序进行讨论：本模型在通过控制其他变量不变，观察半成品在拆解(或直接组装/检测)后，是否对企业生产的总利润造成显著性影响为指标，采取控制变量的方法对单个半成品的拆解问题进行独立化分析，最后得出半成品 3 是否拆解与企业最终的收益无关；由于半成品 1、2 均由 3 个零配件组装而成，关于是否拆解的问题可一起讨论：

在不考虑前期零配件检测问题时，即最低利润情况下，拆解半成品的成本为：

$$W_{\max} = n_0 \cdot \sum_{j=1}^8 x_j + \sum_{r=1}^r W_{rk} \quad , \quad b \leq 3$$

与为减低半成品拆解的机率而考虑零配件检测下所花费的前期投入成本相比可知，零配件是否检测对半成品的是否拆解问题的影响效果与不考虑零配件检测情况对半成品的合格率，无显著性影响，最终本模型得出的结论为：半成品组无需在组装成品前进行拆解，只需进行检验以保障最终成品的合格率，避免进行损耗量更高的调换工序。

最后，由于题中并未给出具体的零配件总件数，和具体的工序总阶段，为避免企业购买的零配件总数对最终决策结果有所影响，本模型以步长为 100，探究了零配件总数从 100~1000 件的所有情况，得出最后结论为：企业购买的零配件总量对生产电子产品的工序无直接影响。

5.3.4 检验与分析

5.3.4.1 扰动检验

首先，选取对最优决策影响较大的几个关键参数(零配件的次品率、检测成本、装配成本等)进行扰动检验，分析成品次品率对生产总成本的影响：

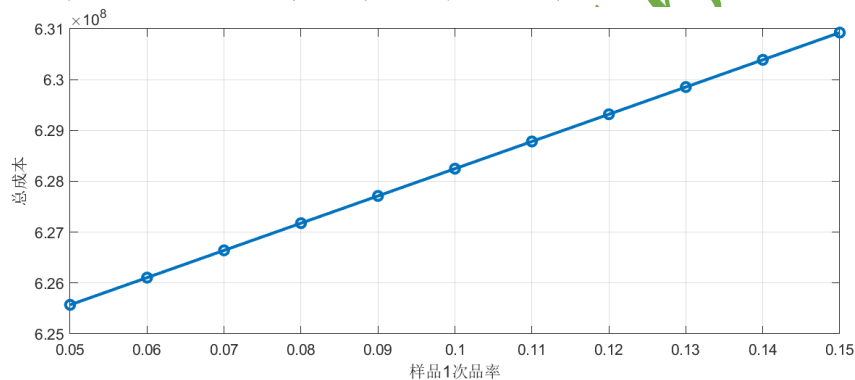


图 12 次品率对总成本的影响分析图

通过分析成品次品率对生产总成本的影响，可以更加深入的剖析次品率如何影响整体成本结构。接着，量化次品率在波动范围内所导致的总成本变化，归结于企业所得利润，为企业提供更加可控的商品生产工序，最大可能的减低产品的次品率，扩大企业收益。

但由于讨论扰动检验的检测范围有限，可能造成检验结果的局限性，为此本模型应根据所求的最优生产决策，进行灵敏度分析，以评估次品率对总成本的影响。在扰动检验过程中，设定成品的次品率在5%至15%的范围内不停波动，以步长为1%，计算总成本对成品次品率变化的连续性灵敏度,如下图：

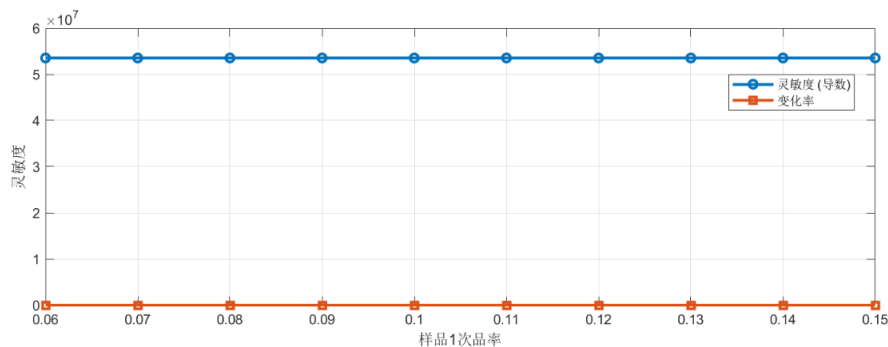


图 13 总成本受次品率的扰动分析图

最终，基于扰动检验结果可知，在不同次品率下，灵敏度波动范围较小，由此可针对不同参数变化情况进行策略调整，以增强模型在实际应用中的适应性和可靠性，由此为企业制定更稳定、高效的生产决策策略。

5.3.5小结

本题针对多工序多阶段的电子产品生产流程进行分析，深入讨论了在不同工序过程中是否检测、拆解、调换的具体抉择，通过引入子工序的方法，分析了零配件、半成品、成品在组装过程中的次品率和所剩余量，挖掘了各道工序的相关性，为此设计了多种检测策略组合。最后为筛查出何种策略最为符合企业生产，以单个半成品为阶段性变量，充分考虑了不同策略下的生产成本与利润等市场性因素，评估各策略的最大经济效益。在评估过程中，采取穷举算法遍历所有企业可能采取的检测与拆解策略，通过对比最终利润的方式，选取最优的决策方案。

5.4问题四模型的建立与求解

5.4.1模型的建立

5.4.1.1抽样检测仿真模型

5.4.1.1.1二项分布下的次品率估计

本题的二项式下的次品率估计运用贝叶斯更新^[11]进行仿真。根据问题1的可知，零配件次品数服从二项分布，即：

$$x \sim B: n(n, P) \quad (73)$$

本文假设 k 为总次品数， x 为次品， n 为检测样本量， P 为次品率， θ 表示待估计的次品率， D 表示抽样检测的观测数据。通过计算似然函数和先验概率密度函数，再带入贝叶斯更新公式，计算给定检测数据后，次品率的分布情况。

1、似然函数

似然函数是一个统计学和机器学习中常出现的概率，用于描述给定参数下，观察到某一数据样本的可能性，在本题中表示在次品率为 θ 的情况下，观测到数据 D 的概率，即：

$$P(D|\theta) = P(x=k|\theta) = C(n, k) \cdot \theta^k \cdot (1-\theta)^{n-k} \quad (74)$$

式中， $C(n, k)$ 表示从 n 个不同项目中选择 k 个方法数。

2、先验概率密度函数

先验概率密度函数用于描述一个未知参数在先验知识或信息的基础上的可能取值的概率密度函数。本题中表示检测之前的次品率分布：

$$P(\theta) = U([\min_{\text{defectrate}}, \max_{\text{defectrate}}]) \quad (75)$$

式中， U 表示随机分布。

3、后验函数

后验函数是贝叶斯统计的基本概念，表示在观察到新数据后对参数的更新。在本题中反映的是给定检测数据 D 后的次品率的概率分布，即：

$$P(\theta|D) = \frac{P(D|\theta) \cdot P(\theta)}{P(D)} = \frac{P(D)}{P(D)} = \frac{\int U([\min_{\text{defectrate}}, \max_{\text{defectrate}}]) \cdot C(n, k) \cdot \theta^k \cdot (1-\theta)^{n-k}}{P(D)} \quad (76)$$

式中， $P(D)$ 表示边缘概率，即先验概率与似然函数的乘积在整个参数空间上的积分。

5.4.1.1.2混合整数规划模型

在问题二和问题三中，采用0-1整数规划模型进行基础建模，但是，在本问中需要运用随机抽样检测进行运算，不存在0-1规划的情况，模型表示为混合整数模型。在问

题二的问题三的基础上，运用后验概率表示花费成本。

更新问题二各个阶段花费成本为：

$$W_r = \begin{cases} \sum_{i=1}^4 COST_i \cdot P(\theta | D_1) \cdot n + COST_2 \cdot P(\theta | D_2) \cdot n (\text{第一阶段}) \\ P(\theta | D_1) \cdot COST_3 \cdot m_j (\text{第二阶段}) \\ P(\theta | D_1) \cdot COST_4 \cdot (m_j - S + S \cdot p_3 \cdot K_i) (\text{第三阶段}) \\ COST_5 \cdot (m_j - s) (\text{第四阶段}) \end{cases} \quad (77)$$

式中， K_i 表示是否进行成品检测。

总成本函数为：

$$W = W_1 + W_2 + W_3 + W_4 \quad (78)$$

即：

$$W = \sum_{i=1}^4 COST_i \cdot P(\theta | D_1) \cdot n + COST_2 \cdot P(\theta | D_2) \cdot n + P(\theta | D_1) \cdot COST_3 \cdot m_j + P(\theta | D_1) \cdot COST_4 \cdot (m_j - S + S \cdot p_3 \cdot K_i) + COST_5 \cdot (m_j - s) \quad (79)$$

更新问题三多工序多阶段花费成本为：

$$W_r = \begin{cases} COST_1 \cdot P(\theta | D) \cdot n_1 + COST_2 \cdot P(\theta | D) \cdot n_1 + COST_3 \cdot P(\theta | D) \cdot n_1 (\text{检测成本}) \\ P(\theta | D) \cdot COST_5 \cdot (m_1 - S_1) (\text{半成品拆解费用}) \\ \sum_{k=1}^3 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}) + \sum_{k=1}^2 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}), r=1 (\text{半成品组装费用}) \\ P(\theta | D) \cdot COST_5 \cdot (m - S), r=1, 2, 3, \dots, k (\text{未检测半成品拆解费用}) \\ P(\theta | D) \cdot COST_5 \cdot (S_j - g + g \cdot Q_2), r=1, 2, 3, \dots, k (\text{检测后半成品拆解费用}) \end{cases} \quad (80)$$

总成本函数为：

$$W = COST_1 \cdot P(\theta | D) \cdot n_1 + COST_2 \cdot P(\theta | D) \cdot n_1 + COST_3 \cdot P(\theta | D) \cdot n_1 + P(\theta | D) \cdot COST_5 \cdot (m_1 - S_1) + \sum_{k=1}^3 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}) + \sum_{k=1}^2 (\sum_{t=1}^{t=2} W_{tk} + W_{rk}) + P(\theta | D) \cdot COST_5 \cdot (m - S) + P(\theta | D) \cdot COST_5 \cdot (S_j - g + g \cdot Q_2), r=1, 2, 3, \dots, k \quad (81)$$

式中， Wr_k 表示符合条件的半成品拆解费用， Wtk 表示组装半成品前期过程花费的检测成本，且 $Wr_k = P(\theta | D) \cdot COST_5 \cdot (m_1 - S_1)$ 、 $Wtk = P(\theta | D) \cdot COST_4 \cdot m_1$ 。

5.4.2模型的求解

5.4.2.1鲸鱼算法

本题采用鲸鱼算法^[12]，在已知问题二决策依据和问题三决策依据的前提下，求解混合整数规划模型。

鲸鱼算法是通过最佳搜索代理或随机变动方式来模拟座头鲸的捕猎行为，通过螺旋路径模拟鲸鱼攻击机制，具有易于实现和搜索性能好的优点。结合本题，将割平面法和分支定界法鲸鱼算法相融合。割平面法工作原理是运用引入的决策依据判断所得方案是否为最优解，舍去非最优解。分支定界法的工作原理是将决策变量的所有可能值划分为不同的分支，并对每个分支进行探索。如果某个分支无法找到最优解，则提前剪枝，避免在该分支上继续耗时搜索，直到寻找到最优解为止。

首先，根据叶贝斯更新公式，求得次品率，为计算鲸鱼算法提供准确的数据支撑。

表 9 问题二次品率更新

情况	零部件 1 更新次品率	零部件 2 更新次品率	成品更新次品率
1	0.11538	0.21154	0.17308
2	0.11538	0.21154	0.17308
3	0.11538	0.21154	0.17308
4	0.11538	0.21154	0.17308
5	0.11538	0.21154	0.17308
6	0.11538	0.21154	0.17308

表 10 问题三次品率更新

部件	次品率	部件	次品率
零配件1	0.11538	零配件8	0.096154
零配件2	0.21154	半成品1	0.13462
零配件3	0.17308	半成品2	0.19231
零配件4	0.076923	半成品3	0.11538
零配件5	0.15385	成品	0.15385
零配件6	0.057692		
零配件7	0.13462		

鲸鱼算法^[13]主要可以分为搜索觅食、气泡网捕食和随机搜索这3个阶段，本题设置鲸鱼数量为10，最大迭代次数为100，作为初始参数，参与计算。

1、搜索觅食阶段。鲸鱼作为独立的个体在一定区域内寻找食物，此时，可以根据鲸鱼的位置进行随机搜索，模型表示为：

$$C = |BX_{rand}(t) - X(t)| \quad (82)$$

$$X(t+1) = X_{rand}(t) - A|B - X_{rand}(t) - X(t)| \quad (83)$$

式中， $X_{rand}(t)$ 表示随机选择的鲸鱼个数， $X(t)$ 表示鲸鱼的具体位置。同时， A 和 B 是系数向量，表示为：

$$A = 2ar_1 - a \quad (84)$$

$$B = 2r_2 \quad (85)$$

式中， a 表示控制参数， r_1 、 r_2 的取值范围均在 $[0,1]$ 之间。

2、气泡网捕食

在此阶段中，鲸鱼通过螺旋更新位置和收缩包围进行捕食。为了更好的模拟和展现这一过程，引入概率 p 在两者之间选择。当 $p < 0.5$ 时，鲸鱼收缩包围；当 $p \geq 0.5$ 时，搜索代理通过螺旋模型更新位置，模型表示为：

$$X(t+1) = D \cdot e^{bl} \cos(2\pi l) + X^*(t) \quad (86)$$

式中， b 为对数螺旋形状常数； l 为区间 $[-1,1]$ 中的随机数。

3、随机搜索

当 $|A| \geq 1$ 时，搜索代理根据彼此位置进行全局搜索，随机选择当前种群中搜索代理，表示为 $X_{rand}(t)$ ，随机搜索公式表示为：

$$X(t+1) = X_{rand}(t) - AD \quad (87)$$

通过运用鲸鱼算法对上述模型进行求解，更新问题二和问题三的结果。

更新后的问题二各个情况的最优方案如下表所示：

表 11 更新问题二结果

情况	方案	是否检测零 部件1	是否检测零 部件2	是否检测成 品	是否拆解不 合格品	成本
1	13	否	否	是	是	3692.5
2	13	否	否	是	是	3692.5
3	13	否	否	是	是	3692.5
4	13	否	否	是	是	3550.2
5	13	否	否	是	是	3550.2
6	14	否	否	是	否	3100

上表是更新之后的问题二各个情况的最优决策方案。在计算过程中，以100为初始零部件个数，以100为步长，到1000截止，计算结果显示任意数量(足够多)的零部件的最优决策方案完全相同，因此，本题足够多的零部件数量对最优决策方案没有影响。

更新后问题三结果为：全部零部件和半成品均未检测，半成品1和2未拆解，半成品3拆解，成品检测，成品未拆解，花费总成本8004.4元。

5.4.3 检验与分析

5.4.3.1 对比分析检验

本题使用鲸鱼算法对模型进行求解，但是在运算过程中需要确定好初始参数。由于鲸鱼的数量影响算法收敛速度，导致计算时间增加，同时增加迭代次数会提高算法精度，增加计算时间，决策变量维度的增加会增加问题的复杂性，导致算法难以找到全局最优解。因此，本题需要通过对比分析检验^[14]，确定最适合的鲸鱼个数、迭代次数和决策变量维度等参数。将最合适的参数与本题算法中选取的参数对比，进一步验证算法计算结果的准确度较高。

本题参数设置以10个为鲸鱼数量的初始参数，以10为步长，增加到30个；以50为最大迭代次数的初始参数，以50为步长，增加到150个；以2为维度的初始参数，以2为步长，增加到6个。下表给出了部分不同参数的状态值，具体状态值见附件 t4_test_results.xlsx:

表 12 不同参数的状态值

鲸鱼数量	最大迭代次数	决策维度	零配件1的数量	零配件2的数量	成品是否检测	不合格成品是否拆解
10	50	2	100	100	NAN	NAN
10	50	2	200	200	NAN	NAN
10	50	2	300	300	NAN	NAN
10	50	2	400	400	NAN	NAN
10	50	2	500	500	NAN	NAN
10	50	2	600	600	NAN	NAN
10	50	2	700	700	NAN	NAN
10	50	2	800	800	NAN	NAN
10	50	2	900	900	NAN	NAN
10	50	2	1000	1000	NAN	NAN

上表给出的是当鲸鱼数量为10、最大迭代次数为50、决策维度为2时，各个参数下的状态值。从表中可以看出，此时无法确定出成品是否检验和不合格品是否拆解的状态，导致后续的最优适应度值存在较大误差，准确度较低。通过对不同参数下各个状态值分析，均出现在决策维度为2时，计算最优适应度误差较大的结果。因此，需要将

维度为2的全部状态剔除。
下图是对各个情况下迭代次数与适应度可视化：

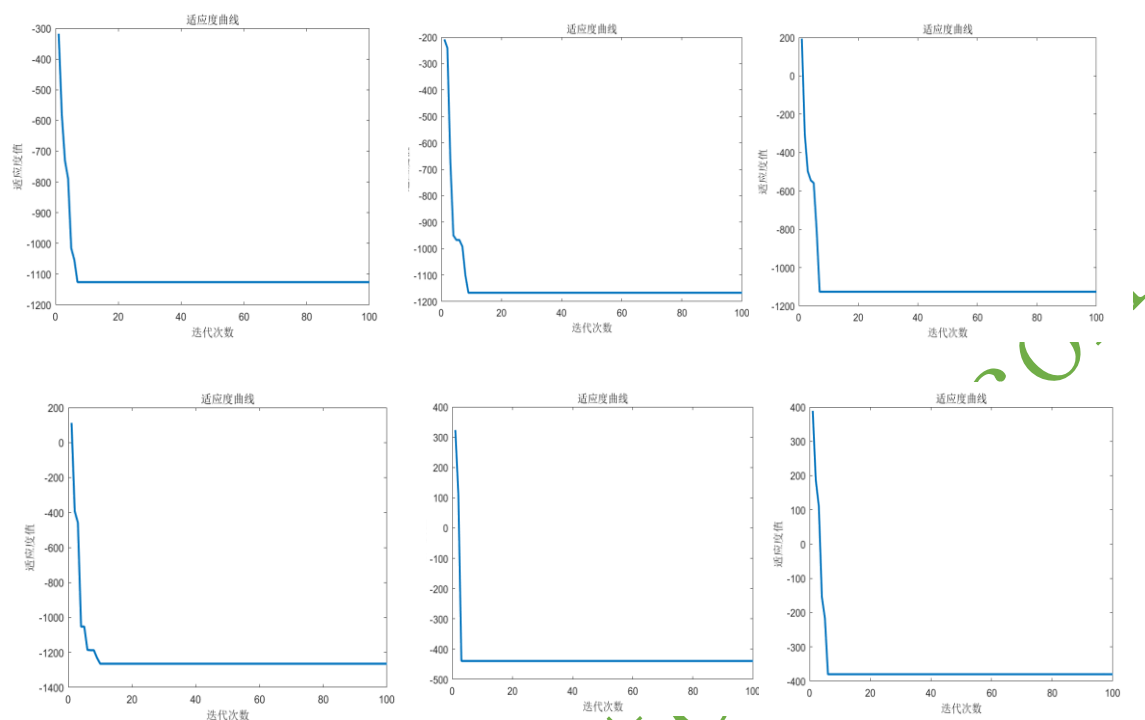


图 14更新问题二的适应性与迭代关系

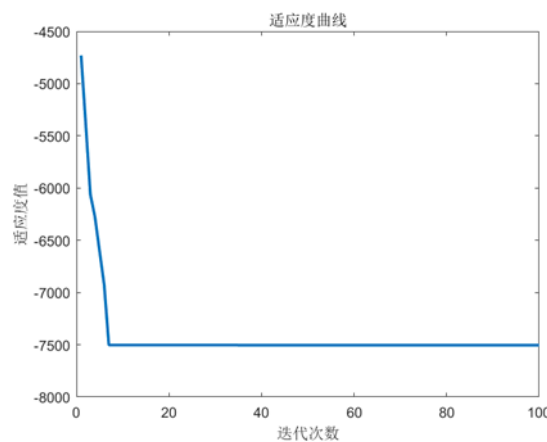


图 15 更新问题三的适应性与迭代关系

本题通过多次对比分析，确定鲸鱼数量为10、算法的最大迭代次数为100、维度为4时，适应度最大，在零配件数量不同的情况下，变化幅度较小，表明此时求解稳定性较高，能够得到个情况下唯一的最优决策方案。本题参数设置与最佳迭代次数和鲸鱼数量保持一致，说明本题算法精度较高。

5.4.4小结

首先，根据题目次品率由抽样检测得到的要求，引入二项式下的贝叶斯更新进行仿真。接着，将后验函数带入问题二和问题三表示出的成本函数，得到更新后的成本函数，建立混合整数规划模型。然后，运用基于割平面法和分支定界法的鲸鱼算法对模型进行求解，更新了问题二(见上表)和问题三的结果。最后，运用对比分析法对算法进行检验，发现最优迭代次数、鲸鱼数量和维度与本题设置参数一致，表明本题算法准确度较高。

六、模型的评价与推广

6.1 模型的评价

在问题一中，运用二项分布模型来模拟零配件的次品率，并巧妙地融合假设检验框架，以科学严谨的方式评估供货商数据的可靠性，从而显著提升了检测结果的精确性。进一步，通过灵敏度分析，验证了模型的稳健性，揭示了在不同信度阈值下，标称次品率的波动对最终结果的影响微乎其微，增强了模型的可靠性与实用性。该模型的核心优势在于其数学表达式的精准与清晰，它有效地揭示了各参数之间的内在联系，不仅便于理论推导，也促进了实际计算的便捷性。然而，模型中涉及的复杂数学公式与假设条件，对使用者提出了较高的专业知识要求，可能成为普及应用的潜在障碍。特别是在处理大规模数据集时，计算复杂度的增加可能导致资源消耗显著，对实际应用构成挑战。鉴于上述局限性，可以考虑探索模型的通用性与可扩展性，特别是将其应用范围从零配件检测拓展至整群抽样、系统抽样等其他抽样检测领域，以满足不同场景下的多样化需求，进一步提升模型的实用价值与影响力。

在问题二中，多阶段决策模型全面覆盖了企业生产的各个环节，从零配件检测到成品检验，再到不合格品的拆解与调换，为企业提供了一站式的决策支持体系。其核心优势在于以最小化总成本为导向，利用穷举法精准锁定最优生产路径，确保决策的精准高效。该模型具备高度灵活性，能根据生产环境的细微变化快速调整决策变量，适应复杂多变的生产需求。然而，值得注意的是，由于模型跨越多个阶段并涉及大量变量的复杂组合，其计算复杂度相应提升，可能导致计算耗时较长。为此，该模型强调初始参数设置的重要性，不合理的参数可能导致结果偏离预期，影响决策的有效性。为解决计算效率问题，推荐采用并行计算等高效算法，这些算法能够显著缩短计算时间，提升整体效率，确保企业快速响应市场变化。此外，为了增强模型的稳定性和可靠性，增加参数敏感性分析显得尤为重要。通过深入分析不同参数设置下的模型表现，可以确保模型在不同条件下均能保持优良的决策能力，进一步提升其鲁棒性和实际应用价值。

在问题三中，基于多阶段决策模型，融入了子工序预测模型，使生产流程中的决策更加细致入微。这一模型专为实际生产中的多样化组合设计，为企业量身定制了具体的决策蓝图和可靠依据。经过扰动检验的严格考验，该模型在不同次品率环境下均展现出卓越的稳定性。然而，值得注意的是，模型涵盖了多个子工序的精密组合与决策变量的复杂交织，这无疑增加了理解和实施的挑战性。为了确保模型的准确性和实用性，该模型强烈依赖于详尽的生产与质量数据作为支撑。但随之而来的是，大量变量与组合情境的处理对计算资源提出了较高要求。为应对这一挑战，可以考虑优化模型结构，剔除冗余复杂度，力求提升模型的可理解性和可操作性。同时，引入机器学习这一强大工具，利用其卓越的数据分析和模式识别能力，显著提升数据处理速度与决策精准度。

在问题四中，抽样检测仿真模型结合贝叶斯更新法精准动态调整次品率，并与混合整数规划模型无缝对接，实现了决策方案的科学飞跃与效能提升。此模型展现出强大的通用性，轻松适配各类抽样方法与检测需求，为企业质量管理提供了前所未有的灵活性与深度。将鲸鱼算法^[15]融入求解过程，不仅显著加速了计算步伐，更确保了结果的精确无误，让复杂数据的处理变得游刃有余。然而，面对大规模数据的洪流，即便拥有如此高效的算法，计算复杂度仍是不可忽视的挑战。值得注意的是，模型效果的基石在于参数的合理设定。不当的参数可能导致结果大幅偏离预期。为此，可以考

虑自动化的参数设定工具，旨在减少人工干预的误差，让模型更加稳健地适应各种应用场景，确保决策始终稳健可靠。

6.2 模型的推广

分层随机抽样检测模型以其灵活性，能够无缝融入产品质量控制的多元抽样策略中，包括整群抽样、系统抽样等，广泛应用于食品安全、药品生产及制造业零部件质量控制等领域。在食品安全领域，它精准抽检样本，全面评估食品质量；在药品生产中，则严格把关成分含量，确保药品合规。对于制造业，该模型通过模拟零部件次品率，为质量稳定筑起坚实防线。

多阶段决策模型，专为复杂系统量身打造，如供应链管理、项目管理及资源分配等场景。在供应链管理中，它助力企业精细化运作，优化库存管理与物流调度，实现成本降低与效率提升的双重目标。项目管理方面，它则成为规划利器，精准分配各阶段任务，确保项目顺利推进。

进一步地，多工序多阶段决策模型在高度复杂的生产环境中大放异彩，如汽车制造与电子产品生产。它深入每个子工序与阶段，优化生产流程，制定最佳决策方案，显著提升生产效率与产品质量，为制造业转型升级提供强大动力。

抽样检测仿真模型，作为市场调研与社会调查的得力助手，通过科学抽样，精准捕捉总体特征，为决策提供坚实数据支撑。在市场调研中，它助力企业洞悉消费者行为，把握需求趋势，为市场营销策略的制定提供宝贵洞见。

综上所述，本文构建的各类模型不仅在生产领域展现出卓越的应用价值，更具备跨行业推广的潜力，为各行各业提供高效、科学的决策与优化方案，推动各行业效率与质量的双重飞跃。

参考文献

- [1] 董方.电子产品制造业供应链韧性评价研究[D].山东理工大学,2023.DOI:10.27276/d.cnki.gsdgc.2023.001094.
- [2] 吕萍.住户调查中户内样本抽样方法的比较研究[J].数理统计与管理,2016,35(01):98-105.DOI:10.13860/j.cnki.sljt.20160122-042.
- [3] Gaurav S ,Chandraketu S .Proficient randomized response model based on blank card strategy to estimate the sensitive parameter under negative binomial distribution[J].Ain Shams Engineering Journal,2022,13(5):
- [4] 王瑞,杨沁雨,欧光龙,等.思茅区森林生物量分层抽样调查设计[J].林业资源管理,2021,(01):197-202.DOI:10.13466/j.cnki.lyzygl.2021.01.025.
- [5] 周杨,邱国新,庄玮玮.密度比模型下高阶随机占优的假设检验(英文)[J].中国科学技术大学学报,2021,51(12):868-878.
- [6] 王雪琴,朱阿元,马吴宁,等.无杆式高压气动弹射器弹射性能参数灵敏度分析[J].南京理工大学学报,2024,48(01):26-35.DOI:10.14177/j.cnki.32-1397n.2024.48.01.002.
- [7] 汤巧英.浅谈离散数学中逻辑推理证明题的解题方法[J].中国人民公安大学学报(自然科学版),1998,(03):12-15.
- [8] 汤炜欢.0-1 整数规划的介绍及其应用[J].经贸实践,2016,(21):247.
- [9] 杨胜彬,伍平,汪国明.基于随机穷举算法最优化污水处理站建造方案[J].华东师范大学学报(自然科学版),2015,(03):21-30+56.

-
- [10] 杨鑫刚,王起全,陈文涛,等.基于蒙特卡洛法的机械制造企业安全绩效评估研究[J].西安科技大学学报,2022,42(06):1189-1197.DOI:10.13800/j.cnki.xakjdxxb.2022.0617.
- [11] 仇文岗,顾鑫,刘汉龙,等.基于贝叶斯更新的非饱和土坡参数概率反演及变形预测[J].岩土力学,2022,43(04):1112-1122.DOI:10.16285/j.rsm.2021.1150.
- [12] 程萱,付光杰,张晓莹,等.基于改进鲸鱼算法虚拟电厂经济优化调度[J].电子设计工程,2022,30(08):161-165.DOI:10.14022/j.issn1674-6236.2022.08.034.
- [13] Razzaq H H ,AlRammahi H M F L ,Mahdi M A .Modified Whale Algorithm and Morley PSO-ML-Based Hyperparameter Optimization for Intrusion Detection[J].International Journal of Image and Graphics,2024,(prepublish):
- [14] E. J ,V. M ,A. R , et al.AI-enabled Cardiac Chambers Volumetry In Non-contrast Cardiac CT Scans Detects HfREF Vs. HfPEF[J].Journal of Cardiovascular Computed Tomography,2023,17(4S):S62-S63.
- [15] 王颖超.鲸鱼优化算法研究与应用进展[J].计算机工程与科学,2024,46(05):881-896.

附录	
 t1_cipingailvfenbu	问题一：次品概率分布绘图代码
 t1_lingmingdu	问题一：灵敏度分析代码
 t1_qiujie	问题一：模型求解代码
 t2_jianyan	问题二：模型检验代码
 t2_qiujie	问题二：模型求解代码
 t3_jianyan	问题三：模型检验代码
 t3_qiujie	问题三：模型求解代码
 t4_2_xunlian	问题四：对问题三的鲸鱼算法进行训练代码
 t4_1_beiyasi	问题四：对问题二贝叶斯更新代码
 t4_1_jianyan	问题四：对问题二的鲸鱼算法检验代码
 t4_1_qiujie	问题四：对问题二的情况进行求解代码
 t4_1_xunlian	问题四：对问题三的鲸鱼算法进行训练代码
 t4_2_beiyasi	问题四：对问题三贝叶斯更新代码
 t4_2_qiujie	问题四：对问题三的情况进行求解代码
 t2_results	问题二：不同零配件数量下的决策方案比较
 t3_results	问题三：不同零配件数量下的决策方案比较
 t4_test_results	问题四：对鲸鱼算法的参数检验

问题一代码

问题一

模型求解代码(MATLAB)

```
% 第一问求解代码
% 运行后的结果为:
% 在 95%信度下, 所需的最小样本量为: 98
% 在 90%信度下, 所需的最小样本量为: 59

clc;
clear;

% 输入参数
p0 = 0.10; % 标称次品率
epsilon_range = 0.01:0.01:0.05; % 允许误差范围

% Z 值(正态分布临界值对应 95% 和 90% 的信度)
Z_95 = 1.645; % 对应 95% 信度的 Z 值
Z_90 = 1.28; % 对应 90% 信度的 Z 值

% 初始化结果数组
results_95 = zeros(size(epsilon_range));
results_90 = zeros(size(epsilon_range));

% 遍历不同的允许误差
for i = 1:length(epsilon_range)
    epsilon = epsilon_range(i);

    % 样本量计算公式
    n_95 = (Z_95^2 * p0 * (1 - p0)) / (epsilon^2);
    n_90 = (Z_90^2 * p0 * (1 - p0)) / (epsilon^2);

    % 向上取整, 确保样本量为整数
    n_95 = ceil(n_95);
    n_90 = ceil(n_90);

    % 存储结果
    results_95(i) = n_95;
    results_90(i) = n_90;
end

% 输出结果
for i = 1:length(epsilon_range)
    fprintf('在 95%%信度下, 允许误差为 %.2f 时, 所需的最小样本量为: %d\n',
        epsilon_range(i), results_95(i));
    fprintf('在 90%%信度下, 允许误差为 %.2f 时, 所需的最小样本量为: %d\n',
        epsilon_range(i), results_90(i));
end
```

```
fprintf('\n')
end
```

问题一

绘制次品概率分布图(MATLAB)

% 第一问绘图代码:
% 在 95% 信度下, 最小样本量为 98 时的, 次品概率分布情况, 最高点时的 $Y=13.1557$
% 在 90% 信度下, 最小样本量为 59 时的, 次品概率分布情况, 最高点时的 $Y=10.1686$

```
clc;
clear;
% 设置中文字体
set(0, 'DefaultAxesFontName', 'SimHei'); % 用黑体显示中文
set(0, 'DefaultTextFontName', 'SimHei');
set(0, 'DefaultAxesFontSize', 12);
set(0, 'DefaultTextFontSize', 12);
```

```
% 定义参数
p0 = 0.10; % 标称次品率
n_95 = 98; % 95% 信度下的样本量
n_90 = 59; % 90% 信度下的样本量
Z_95 = 1.645; % 对应 95% 信度的 Z 值
Z_90 = 1.28; % 对应 90% 信度的 Z 值
```

```
% 计算拒绝域的上下限
x_95 = p0 + Z_95 * sqrt(p0 * (1 - p0) / n_95);
x_90 = p0 + Z_90 * sqrt(p0 * (1 - p0) / n_90);
```

```
% 生成 x 轴样本点(次品率范围)
x = linspace(0, 0.2, 1000);
```

```
% 正态分布概率密度函数
pdf_95 = normpdf(x, p0, sqrt(p0 * (1 - p0) / n_95));
pdf_90 = normpdf(x, p0, sqrt(p0 * (1 - p0) / n_90));
```

```
% 绘图
figure;
hold on;
```

```
% 绘制 95% 信度下的概率分布曲线
plot(x, pdf_95, 'k', 'DisplayName', '95% 置信水平');
```

```
% 绘制 90% 信度下的概率分布曲线
```

```

plot(x, pdf_90, 'k--', 'DisplayName', '90% 置信水平');

fill([x_95, x(x > x_95), 0.2], [0, pdf_95(x > x_95), 0], 'b', 'FaceAlpha', 0.3, 'DisplayName',
'95% 拒绝区域');
fill([0, x(x < x_90), x_90], [0, pdf_90(x < x_90), 0], 'r', 'FaceAlpha', 0.3, 'DisplayName', '90%
接收区域');

% 绘制标称次品率 p0 的垂直线
xline(p0, 'k--', 'DisplayName', sprintf('标称次品率 p0 = %.2f', p0));

% 图例
legend('Location', 'best');

% 标题和标签
xlabel('次品率 (p)');
ylabel('概率密度');

% 显示图像
grid on;
hold off;

```

问题一

灵敏度分析(MATLAB)

% 第一问灵敏度分析代码：
 % 探究不同次品率下所需要最小样本量，次品率变化范围0.01~0.2，步长为0.01
 % 计算灵敏度，即所有实际为阳性的样本中，被正确识别为阳性的比例

```

clc;
clear;
% 计算样本量n在不同信度下的值
% 输入参数
p0_range = linspace(0.01, 0.2, 19); % 标称次品率范围
epsilon = 0.05; % 允许误差

```

```

% Z 值(正态分布临界值对应 95% 和 90% 的信度)
Z_95 = 1.645; % 对应 95% 信度的 Z 值
Z_90 = 1.28; % 对应 90% 信度的 Z 值

```

```

% 初始化结果数组
n_95_array = zeros(size(p0_range));
n_90_array = zeros(size(p0_range));

```

```

% 遍历不同的标称次品率，计算对应的最小样本量
for i = 1:length(p0_range)
    p0 = p0_range(i);
    n_95_array(i) = (Z_95^2 * p0 * (1 - p0)) / (epsilon^2);

```

```

n_90_array(i) = (Z_90^2 * p0 * (1 - p0)) / (epsilon^2);
n_95_array(i) = ceil(n_95_array(i));
n_90_array(i) = ceil(n_90_array(i));
end

% 计算灵敏度指标
sensitivity_95 = n_95_array ./ (n_95_array + n_90_array);
sensitivity_90 = n_90_array ./ (n_95_array + n_90_array);

% 可视化结果
figure;
plot(p0_range, sensitivity_95, 'k-', 'LineWidth', 1);
hold on;
plot(p0_range, sensitivity_90, 'r--', 'LineWidth', 2);
xlabel('标称次品率');
ylabel('灵敏度');
legend('95% 置信水平', '90% 置信水平');
grid on;

% 可视化最小样本量
figure;
plot(p0_range, n_95_array, 'k-', 'LineWidth', 1);
hold on;
plot(p0_range, n_90_array, 'r--', 'LineWidth', 2);
xlabel('标称次品率');
ylabel('最小样本量');
legend('95% 置信水平', '90% 置信水平');
grid on;

```

问题二代码

问题二

模型求解(MATLAB)

```

%% 第二问的求解代码
clc; % 清除命令窗口
clear; % 清除工作区变量

%% 定义情况的参数
cases = [
    %defect_rate1 为零配件 1 的次品率, defect_rate2 为零配件 2 的次品率
    %defect_rate_final 为成品次品率

    %inspection_cost1 为零配件 1 的检测成本, inspection_cost2 为零配件 2 的检测成本
    %inspection_cost_final 为成品的检测成本

    %assembly_cost 为成品的装配成本, market_price 为成品的市场售价

```

```

%replacement_loss 为不合格成品的调换损失, dismantle_cost 为不合格成品的拆解成品

    struct('defect_rate1', 0.1, 'defect_rate2', 0.1, 'defect_rate_final', 0.1, 'inspection_cost1', 2,
'inspection_cost2', 3, ...
        'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 6,
'dismantle_cost', 5);
    struct('defect_rate1', 0.2, 'defect_rate2', 0.2, 'defect_rate_final', 0.2, 'inspection_cost1', 2,
'inspection_cost2', 3, ...
        'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 6,
'dismantle_cost', 5);
    struct('defect_rate1', 0.1, 'defect_rate2', 0.1, 'defect_rate_final', 0.1, 'inspection_cost1', 2,
'inspection_cost2', 3, ...
        'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 30,
'dismantle_cost', 5);
    struct('defect_rate1', 0.2, 'defect_rate2', 0.2, 'defect_rate_final', 0.2, 'inspection_cost1', 1,
'inspection_cost2', 1, ...
        'inspection_cost_final', 2, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 30,
'dismantle_cost', 5);
    struct('defect_rate1', 0.1, 'defect_rate2', 0.2, 'defect_rate_final', 0.1, 'inspection_cost1', 8,
'inspection_cost2', 1, ...
        'inspection_cost_final', 2, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 10,
'dismantle_cost', 5);
    struct('defect_rate1', 0.05, 'defect_rate2', 0.05, 'defect_rate_final', 0.05, 'inspection_cost1', 2,
'inspection_cost2', 3, ...
        'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 10,
'dismantle_cost', 40);
];

%% 生成所有策略
strategies = []; % 初始化策略数组
for detect_parts1 = [true, false] % 遍历是否检测零配件 1
    for detect_parts2 = [true, false] % 遍历是否检测零配件 2
        for detect_final = [true, false] % 遍历是否检测成品
            for dismantle = [true, false] % 遍历是否拆解不合格成品
                % 将当前策略组合添加到策略数组中
                strategies = [strategies; struct('detect_parts1', detect_parts1, 'detect_parts2',
detect_parts2, ...
                    'detect_final', detect_final, 'dismantle', dismantle)];
            end
        end
    end
end

% 生成策略解释
strategy_explanations = cell(size(strategies)); % 初始化策略解释数组
for i = 1:length(strategies) % 遍历所有策略
    strategy = strategies(i); % 获取当前策略
    % 生成当前策略的解释字符串

```

```

    strategy_explanations{i} = sprintf('方案 %d: 检测零配件 1: %s, 检测零配件 2: %s, 检测成品: %s, 拆解不合格成品: %s', ...
        i, string(strategy.detect_parts1), string(strategy.detect_parts2), string(strategy.detect_final),
        string(strategy.dismantle));
end

% 定义数量范围
n_range = 100:100:1000;

% 初始化结果表格
results = cell(length(cases) * length(n_range) * length(strategies), 13);
row_index = 1;

%% 计算每个情况下的总成本和次品率
for i = 1:length(cases) % 遍历所有情况
    fprintf('\n 情况 %d:\n', i); % 打印当前情况编号
    for n = n_range % 遍历数量范围
        fprintf('数量: %d\n', n); % 打印当前数量
        for j = 1:length(strategies) % 遍历所有策略
            % 计算当前情况和策略下的总成本和次品率
            [total_cost, defective_rate, cost_1, cost_2, loss_1, loss_2, cost_final, loss_final, cost_3] =
            calc_total_cost(cases(i), strategies(j).detect_parts1, strategies(j).detect_parts2, ...
                strategies(j).detect_final, strategies(j).dismantle, n);

            % 存储结果到表格
            results{row_index, 1} = i; % 情况编号
            results{row_index, 2} = n; % 数量
            results{row_index, 3} = total_cost; % 总成本
            results{row_index, 4} = defective_rate; % 次品率
            results{row_index, 5} = cost_1; % 零配件 1 的检测成本
            results{row_index, 6} = cost_2; % 零配件 2 的检测成本
            results{row_index, 7} = loss_1; % 零配件 1 的损失成本
            results{row_index, 8} = loss_2; % 零配件 2 的损失成本
            results{row_index, 9} = cost_final; % 成品的检测成本
            results{row_index, 10} = loss_final; % 成品的损失成本
            results{row_index, 11} = cost_3; % 拆解不合格成品的成本
            results{row_index, 12} = defective_rate; % 次品率
            results{row_index, 13} = strategy_explanations{j}; % 策略解释
            row_index = row_index + 1;
            fprintf('%s: 总成本 = %.2f, 零配件 1 的检测成本 = %.2f, 零配件 2 的检测成本 = %.2f, 零
            配件 1 的损失成本 = %.2f, 零配件 2 的损失成本 = %.2f, 成品的检测成本 = %.2f, 成品的损失
            成本 = %.2f, 拆解不合格成品的成本: %.2f, 次品率 = %.2f\n', ...
                strategy_explanations{j}, total_cost, cost_1, cost_2, loss_1, loss_2, cost_final, loss_final,
                cost_3, defective_rate);
        end
    end
end
end

```

```

%% 将结果转换为表格
results_table = cell2table(results, 'VariableNames', {'Case', 'Quantity', 'TotalCost', 'DefectiveRate1', ...
    'Cost1', 'Cost2', 'Loss1', 'Loss2', 'CostFinal', 'LossFinal', 'Cost3', 'DefectiveRate2', 'StrategyExplanation'});

% 将结果保存到 Excel 文件
filename = 't2_results.xlsx';
writetable(results_table, filename);

%% 计算总成本的函数
function [total_cost, defective_rate, cost_1, cost_2, loss_1, loss_2, cost_final, loss_final, cost_3] =
calc_total_cost(current_case, detect_parts1, detect_parts2, detect_final, dismantle, n)
    n_1 = n; % 零配件 1 的数量
    n_2 = n; % 零配件 2 的数量
    n_final_products = n; % 成品数量
    money = n_1 * 4 + n_2 * 18; % 零配件 1、2 的购入成本

    cost_1 = n_1 * current_case.inspection_cost1 * detect_parts1; % 零配件 1 的检测成本
    cost_2 = n_2 * current_case.inspection_cost2 * detect_parts2; % 零配件 2 的检测成本

    loss_1 = (n_1 * current_case.defect_rate1) * current_case.assembly_cost * ~detect_parts1; % 零配件 1 的损失成本
    loss_2 = (n_2 * current_case.defect_rate2) * current_case.assembly_cost * ~detect_parts2; % 零配件 2 的损失成本

    cost_final = n_final_products * current_case.inspection_cost_final * detect_final; % 成品的检测成本
    loss_final = (n_final_products * current_case.defect_rate_final) * current_case.replacement_loss * ~detect_final; % 成品的损失成本

    cost_3 = n_final_products * current_case.dismantle_cost * dismantle; % 拆解不合格成品的成本

    dismantle_revenue = (n_final_products * current_case.defect_rate_final) * current_case.market_price * dismantle; % 拆解不合格成品的收入

    total_cost = money + cost_1 + cost_2 + loss_1 + loss_2 + cost_final + loss_final + cost_3; % 总成本
    defective_rate = current_case.defect_rate_final * (1 - detect_final * 0.5); % 次品率
end

```

问题二

蒙特卡洛检验(MATLAB)

```

%% 第二问蒙特卡洛检验代码
clc; % 清除命令窗口
clear; % 清除工作区变量

%% 定义情况的参数
cases = [
    %defect_rate1 为零配件 1 的次品率, defect_rate2 为零配件 2 的次品率
    %defect_rate_final 为成品次品率

    %inspection_cost1 为零配件 1 的检测成本, inspection_cost2 为零配件 2 的检测成本
    %inspection_cost_final 为成品的检测成本

    %assembly_cost 为成品的装配成本, market_price 为成品的市场售价
    %replacement_loss 为不合格成品的调换损失, dismantle_cost 为不合格成品的拆解成本

    struct('defect_rate1', 0.1, 'defect_rate2', 0.1, 'defect_rate_final', 0.1, 'inspection_cost1', 2,
    'inspection_cost2', 3, ...
    'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 6,
    'dismantle_cost', 5);
    struct('defect_rate1', 0.2, 'defect_rate2', 0.2, 'defect_rate_final', 0.2, 'inspection_cost1', 2,
    'inspection_cost2', 3, ...
    'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 6,
    'dismantle_cost', 5);
    struct('defect_rate1', 0.1, 'defect_rate2', 0.1, 'defect_rate_final', 0.1, 'inspection_cost1', 2,
    'inspection_cost2', 3, ...
    'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 30,
    'dismantle_cost', 5);
    struct('defect_rate1', 0.2, 'defect_rate2', 0.2, 'defect_rate_final', 0.2, 'inspection_cost1', 1,
    'inspection_cost2', 1, ...
    'inspection_cost_final', 2, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 30,
    'dismantle_cost', 5);
    struct('defect_rate1', 0.1, 'defect_rate2', 0.2, 'defect_rate_final', 0.1, 'inspection_cost1', 8,
    'inspection_cost2', 1, ...
    'inspection_cost_final', 2, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 10,
    'dismantle_cost', 5);
    struct('defect_rate1', 0.05, 'defect_rate2', 0.05, 'defect_rate_final', 0.05, 'inspection_cost1', 2,
    'inspection_cost2', 3, ...
    'inspection_cost_final', 3, 'assembly_cost', 6, 'market_price', 56, 'replacement_loss', 10,
    'dismantle_cost', 40);
];

%% 生成所有策略
strategies = []; % 初始化策略数组
for detect_parts1 = [true, false] % 遍历是否检测零配件 1
    for detect_parts2 = [true, false] % 遍历是否检测零配件 2
        for detect_final = [true, false] % 遍历是否检测成品

```

```

        for dismantle = [true, false] % 遍历是否拆解不合格成品
            % 将当前策略组合添加到策略数组中
            strategies = [strategies; struct('detect_parts1', detect_parts1, 'detect_parts2',
detect_parts2, ...
                                            'detect_final', detect_final, 'dismantle', dismantle)];
        end
    end
end
end

% 生成策略解释
strategy_explanations = cell(size(strategies)); % 初始化策略解释数组
for i = 1:length(strategies) % 遍历所有策略
    strategy = strategies(i); % 获取当前策略
    % 生成当前策略的解释字符串
    strategy_explanations{i} = sprintf('方案 %d: 检测零配件 1: %s, 检测零配件 2: %s,
检测成品: %s, 拆解不合格成品: %s', ...
        i, string(strategy.detect_parts1), string(strategy.detect_parts2),
string(strategy.detect_final), string(strategy.dismantle));
end

% 定义数量范围
n_range = 100:100:1000;

% 初始化结果表格
results = cell(length(cases) * length(n_range) * length(strategies), 6);
row_index = 1;

%% 计算每个情况下的总成本和次品率
for i = 1:length(cases) % 遍历所有情况
    fprintf('\n 情况 %d:\n', i); % 打印当前情况编号
    for n = n_range % 遍历数量范围
        fprintf('数量: %d\n', n); % 打印当前数量
        for j = 1:length(strategies) % 遍历所有策略
            % 计算当前情况和策略下的总成本和次品率
            [total_cost, defective_rate] = calc_total_cost(cases(i), strategies(j).detect_parts1,
strategies(j).detect_parts2, ...
                    strategies(j).detect_final, strategies(j).dismantle, n);
            % 存储结果到表格
            results{row_index, 1} = i; % 情况编号
            results{row_index, 2} = n; % 数量
            results{row_index, 3} = strategy_explanations{j}; % 策略解释
            results{row_index, 4} = total_cost; % 总成本
            results{row_index, 5} = defective_rate; % 次品率
            results{row_index, 6} = strategy_explanations{j}; % 策略解释
            row_index = row_index + 1;
        end
    end
end
end

```

```

        fprintf('%s: 总成本 = %.2f, 次品率 = %.2f\n', strategy_explanations{j}, total_cost,
defective_rate); % 打印当前策略的总成本和次品率
    end
end
end

%% 蒙特卡洛模拟
num_simulations = 1000; % 模拟次数
defective_rates_with_uncertainty = zeros(num_simulations, 1);
defective_rates_without_uncertainty = zeros(num_simulations, 1);

for i = 1:num_simulations
    % 随机选择一个情况和策略
    case_index = randi(length(cases));
    strategy_index = randi(length(strategies));
    n = n_range(randi(length(n_range)));

    % 计算没有随机性的次品率
    [~, defective_rate_without_uncertainty] = calc_total_cost(cases(case_index),
strategies(strategy_index).detect_parts1, ...
strategies(strategy_index).detect_parts2,
strategies(strategy_index).detect_final, ...
strategies(strategy_index).dismantle, n);
    defective_rates_without_uncertainty(i) = defective_rate_without_uncertainty;

    % 计算有随机性的次品率
    [~, defective_rate_with_uncertainty] =
calc_total_cost_with_uncertainty(cases(case_index),
strategies(strategy_index).detect_parts1, ...
strategies(strategy_index).detect_parts2,
strategies(strategy_index).detect_final, ...
strategies(strategy_index).dismantle, n);
    defective_rates_with_uncertainty(i) = defective_rate_with_uncertainty;
end

%% 可视化结果
figure;
subplot(2,1,1);
histogram(defective_rates_without_uncertainty, 'Normalization', 'pdf');
title('未添加随机性的次品率分布');
xlabel('次品率');
ylabel('概率密度');

subplot(2,1,2);
histogram(defective_rates_with_uncertainty, 'Normalization', 'pdf');
title('添加随机性的次品率分布');
xlabel('次品率');

```

```
ylabel('概率密度');
```

```
%% 计算总成本的函数(没有随机性)
```

```
function [total_cost, defective_rate] = calc_total_cost(current_case, detect_parts1,  
detect_parts2, detect_final, dismantle, n)
```

```
    n_1 = n; % 零配件 1 的数量
```

```
    n_2 = n; % 零配件 2 的数量
```

```
    n_final_products = n; % 成品数量
```

```
    money = n_1 * 4 + n_2 * 18; % 零配件 1、2 的购入成本
```

```
    cost_1 = n_1 * current_case.inspection_cost1 * detect_parts1; % 零配件 1 的检测成本
```

```
    cost_2 = n_2 * current_case.inspection_cost2 * detect_parts2; % 零配件 2 的检测成本
```

```
    loss_1 = (n_1 * current_case.defect_rate1) * current_case.assembly_cost *  
~detect_parts1; % 零配件 1 的损失成本
```

```
    loss_2 = (n_2 * current_case.defect_rate2) * current_case.assembly_cost *  
~detect_parts2; % 零配件 2 的损失成本
```

```
    cost_final = n_final_products * current_case.inspection_cost_final * detect_final; % 成品  
的检测成本
```

```
    loss_final = (n_final_products * current_case.defect_rate_final) *  
current_case.replacement_loss * ~detect_final; % 成品的损失成本
```

```
    cost_3 = n_final_products * current_case.dismantle_cost * dismantle; % 拆解不合格成品  
的成本
```

```
    dismantle_revenue = (n_final_products * current_case.defect_rate_final) *  
current_case.market_price * dismantle; % 拆解不合格成品的收入
```

```
    total_cost = money + cost_1 + cost_2 + loss_1 + loss_2 + cost_final + loss_final + cost_3; %  
总成本
```

```
    defective_rate = current_case.defect_rate_final * (1 - detect_final * 0.5); % 次品率  
end
```

```
%% 计算总成本的函数(有随机性)
```

```
function [total_cost, defective_rate] = calc_total_cost_with_uncertainty(current_case,  
detect_parts1, detect_parts2, detect_final, dismantle, n)
```

```
    n_1 = n; % 零配件 1 的数量
```

```
    n_2 = n; % 零配件 2 的数量
```

```
    n_final_products = n; % 成品数量
```

```
    money = n_1 * 4 + n_2 * 18; % 零配件 1、2 的购入成本
```

```
% 引入随机性, 假设次品率服从正态分布
```

```
defect_rate1_random = max(0, min(1, normrnd(current_case.defect_rate1, 0.05)));
```

```
defect_rate2_random = max(0, min(1, normrnd(current_case.defect_rate2, 0.05)));
```

```
defect_rate_final_random = max(0, min(1, normrnd(current_case.defect_rate_final, 0.05)));
```

```

cost_1 = n_1 * current_case.inspection_cost1 * detect_parts1; % 零配件 1 的检测成本
cost_2 = n_2 * current_case.inspection_cost2 * detect_parts2; % 零配件 2 的检测成本

loss_1 = (n_1 * defect_rate1_random) * current_case.assembly_cost * ~detect_parts1; %
零配件 1 的损失成本
loss_2 = (n_2 * defect_rate2_random) * current_case.assembly_cost * ~detect_parts2; %
零配件 2 的损失成本

cost_final = n_final_products * current_case.inspection_cost_final * detect_final; % 成品
的检测成本
loss_final = (n_final_products * defect_rate_final_random) * current_case.replacement_loss
* ~detect_final; % 成品的损失成本

cost_3 = n_final_products * current_case.dismantle_cost * dismantle; % 拆解不合格成品
的成本

dismantle_revenue = (n_final_products * defect_rate_final_random) * current_case.market_price
* dismantle; % 拆解不合格成品的收入

total_cost = money + cost_1 + cost_2 + loss_1 + loss_2 + cost_final + loss_final + cost_3; %
总成本
defective_rate = defect_rate_final_random * (1 - detect_final * 0.5); % 次品率
end

```

问题三代码

问题三

模型求解(MATLAB)

```

%% 问题三的求解代码:
%% 由于数据量较大, 该代码大概会运行 12 分钟左右(电脑环境: 12 核, 线程数 16)

%% 使用结构体定义各个组件、半成品和成品的参数, 包括次品率、采购价格、检测
成本等

% 零部件参数
% defect_rate 为次品率, purchase_price 为购买单价, inspection_cost 为检测成本
clc;
clear;
components = struct(...
    'part1', struct('defect_rate', 0.1, 'purchase_price', 2, 'inspection_cost', 1), ...
    'part2', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...
    'part3', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2), ...
    'part4', struct('defect_rate', 0.1, 'purchase_price', 2, 'inspection_cost', 1), ...
    'part5', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...
    'part6', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2), ...
    'part7', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...
    'part8', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2) ...

```

```

);

% 半成品参数
% defect_rate 为次品率, assembly_cost 为装配成本
% inspection_cost 为检测成本, dismantle_cost 为拆解成本
semi_products = struct(...
    'semi_product1', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4,
    'dismantle_cost', 6), ...
    'semi_product2', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4,
    'dismantle_cost', 6), ...
    'semi_product3', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4,
    'dismantle_cost', 6) ...
);

% 成品参数
% defect_rate 为次品率, assembly_cost 为装配成本, inspection_cost 为检测成本
% dismantle_cost 为拆解费用, selling_price 为市场售价, exchange_loss 为调换损失
final_product = struct(...
    'final_product', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 6,
    'dismantle_cost', 10, 'selling_price', 200, 'exchange_loss', 40) ...
);

%%
% 生成所有可能的组合
% 使用二进制表示法生成所有可能的检测策略组合
component_combinations = dec2bin(0:2^8-1) == '1';
pro_combinations = dec2bin(0:2^3-1) == '1';
final_combinations = dec2bin(0:2^2-1) == '1';

% 初始化结果存储
% 初始化一个空的单元格数组来存储每个策略的结果
results = cell(0, 10);
strategy_number = 1;

% 定义零部件数量范围
component_quantities = 100:100:1000;

%%
% 计算所有策略的成本和次品率
% 遍历所有可能的组合, 计算每个策略的总成本和次品率
for q = 1:length(component_quantities)
    quantity = component_quantities(q);
    for i = 1:size(component_combinations, 1)
        for j = 1:size(pro_combinations, 1)
            for k = 1:size(final_combinations, 1)
                detect_components = component_combinations(i, :);
                detect_semi = pro_combinations(j, :);
                detect_final = final_combinations(k, 1);
            end
        end
    end
end

```

```

    dismantle = final_combinations(k, 2);

    % 调用函数计算当前策略的总成本和次品率
    [cost, defective_rate, component_costs, semi_product_costs, final_product_costs]
= calculate_expected_cost(components, semi_products, final_product, detect_components,
detect_semi, detect_final, dismantle, quantity);

    % 生成当前策略的描述
    strategy_description = generate_strategy_description(detect_components,
detect_semi, detect_final, dismantle);

    % 将策略编号、描述、总成本、次品率、零部件数量、各阶段费用存储到
结果数组中
    results = [results; {strategy_number, strategy_description, cost, defective_rate,
quantity, component_costs, semi_product_costs, final_product_costs}];
    strategy_number = strategy_number + 1;
end
end
end
end

% 将结果保存到 Excel 文件
% 将结果数组转换为表格，并保存到 Excel 文件中
%这里的费用指的是检验+拆解的总费用(如果没有拆解，就只是检验的费用)
results_table = cell2table(results, 'VariableNames', {'策略编号', '策略描述', '总成本', '成品
次品率', '零部件数量', '零部件费用', '半成品费用', '成品费用'});
writetable(results_table, 't3_results.xlsx', 'WriteVariableNames', true);

%%
% 计算预期成本的函数
% 该函数计算给定策略的总成本和次品率
function [total_cost, total_defective_rate, component_costs, semi_product_costs,
final_product_costs] = calculate_expected_cost(components, semi_products, final_product,
detect_components, detect_semi, detect_final, dismantle, quantity)
    total_cost = 0;
    total_defective_rate = 1;
    component_costs = 0;
    semi_product_costs = 0;
    final_product_costs = 0;

    % 计算组件的成本和次品率
    component_names = fieldnames(components);
    for i = 1:length(component_names)
        comp_name = component_names{i};
        comp_data = components.(comp_name);
        purchase_cost = comp_data.purchase_price * quantity;
        inspection_cost = comp_data.inspection_cost * detect_components(i) * quantity;
        cost = purchase_cost + inspection_cost;
    end
end

```

```

defective_rate = comp_data.defect_rate * (1 - 0.5 * detect_components(i));
component_costs = component_costs + cost;
total_cost = total_cost + cost;
total_defective_rate = total_defective_rate * (1 - defective_rate);
end

% 计算半成品的成本和次品率
semi_product_names = fieldnames(semi_products);
for i = 1:length(semi_product_names)
    semi_name = semi_product_names{i};
    semi_data = semi_products.(semi_name);
    assembly_cost = semi_data.assembly_cost * quantity;
    inspection_cost = semi_data.inspection_cost * detect_semi(i) * quantity;
    dismantle_cost = semi_data.dismantle_cost * detect_semi(i) * quantity;
    cost = assembly_cost + inspection_cost + dismantle_cost;
    defective_rate = semi_data.defect_rate * (1 - 0.5 * detect_semi(i));
    semi_product_costs = semi_product_costs + cost;
    total_cost = total_cost + cost;
    total_defective_rate = total_defective_rate * (1 - defective_rate);
end

% 计算成品的成本和次品率
final_defective_rate = final_product.final_product.defect_rate * (0.5 * detect_final + 0.5
* ~detect_final);
assembly_cost = final_product.final_product.assembly_cost * quantity;
inspection_cost = final_product.final_product.inspection_cost * detect_final * quantity;
dismantle_cost = final_product.final_product.dismantle_cost * dismantle * quantity;
final_cost = assembly_cost + inspection_cost + dismantle_cost;

final_product_costs = final_cost;
total_cost = total_cost + final_cost;
total_defective_rate = 1 - (total_defective_rate * (1 - final_defective_rate));

% 计算由于次品率导致的损失
exchange_loss = final_product.final_product.exchange_loss * total_defective_rate *
quantity;
total_cost = total_cost + exchange_loss;

% 如果策略中包含拆解不合格成品，则增加拆解成本
if dismantle
    dismantle_cost = final_product.final_product.dismantle_cost * quantity;
    final_product_costs = final_product_costs + dismantle_cost;
    total_cost = total_cost + dismantle_cost;
end
end

%%
% 生成策略描述的函数
% 该函数生成当前策略的描述字符串

```

```

function description = generate_strategy_description(detect_components, detect_semi,
detect_final, dismantle)
    description = '';
    component_names = {'零配件 1', '零配件 2', '零配件 3', '零配件 4', '零配件 5', '零配件
6', '零配件 7', '零配件 8'};
    for i = 1:length(detect_components)
        if detect_components(i)
            description = [description, sprintf('检测%s, ', component_names{i})];
        else
            description = [description, sprintf('不检测%s, ', component_names{i})];
        end
    end
    semi_product_names = {'半成品 1', '半成品 2', '半成品 3'};
    for i = 1:length(detect_semi)
        if detect_semi(i)
            description = [description, sprintf('检测%s, ', semi_product_names{i})];
        else
            description = [description, sprintf('不检测%s, ', semi_product_names{i})];
        end
    end
    if detect_final
        description = [description, '检测成品, '];
    else
        description = [description, '不检测成品, '];
    end
    if dismantle
        description = [description, '拆解不合格成品'];
    else
        description = [description, '不拆解不合格成品'];
    end
end

```

问题三

蒙特卡洛检验(MATBAL)

```

%% 第三问的检验代码
%% 零部件参数
clc;
clear;
components = struct(...
    'part1', struct('defect_rate', 0.1, 'purchase_price', 2, 'inspection_cost', 1), ...
    'part2', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...
    'part3', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2), ...
    'part4', struct('defect_rate', 0.1, 'purchase_price', 2, 'inspection_cost', 1), ...
    'part5', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...
    'part6', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2), ...
    'part7', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...
    'part8', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2) ...
);

% 半成品参数
semi_products = struct(...
    'semi_product1', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4,
'dismantle_cost', 6), ...
    'semi_product2', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4,
'dismantle_cost', 6), ...
    'semi_product3', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4,
'dismantle_cost', 6) ...
);

% 成品参数
final_product = struct(...
    'final_product', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 6,
'dismantle_cost', 10, 'selling_price', 200, 'exchange_loss', 40) ...
);

% 定义零部件数量范围
component_quantities = 500;

%% 生成所有策略
% 生成所有可能的组合
% 使用二进制表示法生成所有可能的检测策略组合
component_combinations = dec2bin(0:2^8-1) == '1';
semi_combinations = dec2bin(0:2^3-1) == '1';
final_combinations = dec2bin(0:2^2-1) == '1';

% 次品率变化范围
defect_rate_range = 0.05:0.01:0.15;

% 初始化结果存储
total_costs = zeros(length(defect_rate_range), 1);

% 遍历次品率范围

```

```

for d = 1:length(defect_rate_range)
    % 更新 part1 的次品率
    components.part1.defect_rate = defect_rate_range(d);

    % 计算所有策略的成本和次品率
    total_cost = 0;
    for i = 1:size(component_combinations, 1)
        for j = 1:size(semi_combinations, 1)
            for k = 1:size(final_combinations, 1)
                detect_components = component_combinations(i, :);
                detect_semi = semi_combinations(j, :);
                detect_final = final_combinations(k, 1);
                dismantle = final_combinations(k, 2);

                % 调用函数计算当前策略的总成本和次品率
                [cost, ~, ~, ~, ~] = calculate_expected_cost(components, semi_products,
                    final_product, detect_components, detect_semi, detect_final, dismantle,
                    component_quantities);

                % 累加总成本
                total_cost = total_cost + cost;
            end
        end
    end

    % 存储总成本
    total_costs(d) = total_cost;
end

%% 计算灵敏度
% 计算灵敏度(导数)
sensitivity = diff(total_costs) ./ diff(defect_rate_range);

% 计算变化率
change_rate = sensitivity ./ total_costs(1:end-1);

% 可视化结果
figure;
subplot(2,1,1);
plot(defect_rate_range, total_costs, '-o', 'LineWidth', 2);
xlabel('样品 1 次品率');
ylabel('总成本');
grid on;

subplot(2,1,2);
plot(defect_rate_range(2:end), sensitivity, '-o', 'LineWidth', 2);
hold on;
plot(defect_rate_range(2:end), change_rate, '-s', 'LineWidth', 2);
xlabel('样品 1 次品率');

```

```

ylabel('灵敏度');
legend('灵敏度 (导数)', '变化率');
grid on;

%%
% 计算预期成本的函数
% 该函数计算给定策略的总成本和次品率
function [total_cost, total_defective_rate, component_costs, semi_product_costs,
final_product_costs] = calculate_expected_cost(components, semi_products, final_product,
detect_components, detect_semi, detect_final, dismantle, quantity)
    total_cost = 0;
    total_defective_rate = 1;
    component_costs = 0;
    semi_product_costs = 0;
    final_product_costs = 0;

    % 计算组件的成本和次品率
    component_names = fieldnames(components);
    for i = 1:length(component_names)
        comp_name = component_names{i};
        comp_data = components.(comp_name);
        purchase_cost = comp_data.purchase_price * quantity;
        inspection_cost = comp_data.inspection_cost * detect_components(i) * quantity;
        cost = purchase_cost + inspection_cost;
        defective_rate = comp_data.defect_rate * (1 - 0.5 * detect_components(i));
        component_costs = component_costs + cost;
        total_cost = total_cost + cost;
        total_defective_rate = total_defective_rate * (1 - defective_rate);
    end

    % 计算半成品的成本和次品率
    semi_product_names = fieldnames(semi_products);
    for i = 1:length(semi_product_names)
        semi_name = semi_product_names{i};
        semi_data = semi_products.(semi_name);
        assembly_cost = semi_data.assembly_cost * quantity;
        inspection_cost = semi_data.inspection_cost * detect_semi(i) * quantity;
        dismantle_cost = semi_data.dismantle_cost * detect_semi(i) * quantity;
        cost = assembly_cost + inspection_cost + dismantle_cost;
        defective_rate = semi_data.defect_rate * (1 - 0.5 * detect_semi(i));
        semi_product_costs = semi_product_costs + cost;
        total_cost = total_cost + cost;
        total_defective_rate = total_defective_rate * (1 - defective_rate);
    end

    % 计算成品的成本和次品率
    final_defective_rate = final_product.final_product.defect_rate * (0.5 * detect_final + 0.5
* ~detect_final);
    assembly_cost = final_product.final_product.assembly_cost * quantity;

```

```

inspection_cost = final_product.final_product.inspection_cost * detect_final * quantity;
dismantle_cost = final_product.final_product.dismantle_cost * dismantle * quantity;
final_cost = assembly_cost + inspection_cost + dismantle_cost;

final_product_costs = final_cost;
total_cost = total_cost + final_cost;
total_defective_rate = 1 - (total_defective_rate * (1 - final_defective_rate));

% 计算由于次品率导致的损失
exchange_loss = final_product.final_product.exchange_loss * total_defective_rate *
quantity;
total_cost = total_cost + exchange_loss;

% 如果策略中包含拆解不合格成品，则增加拆解成本
if dismantle
    dismantle_cost = final_product.final_product.dismantle_cost * quantity;
    final_product_costs = final_product_costs + dismantle_cost;
    total_cost = total_cost + dismantle_cost;
end
end

```

问题四代码

问题四

贝叶斯更新(MATBAL)

```

%% 代码说明
%通过贝叶斯更新函数，在问题二的背景下，该代码计算零配件 1、配件 2 和成品的
后验概率

%% 数据初始化
clc
clear
%Part1 为零部件 1，Part2 为零部件 2，Product 为成品
% defect_rate 为次品率    purchase_price 为购买单价
% inspection_cost 为检测成本 assembly_cost 为装配成本
ifs = struct(...
    'Scenario1', struct('Part1', struct('defect_rate', 0.1, 'purchase_price', 4, 'inspection_cost',
2), ...
        'Part2', struct('defect_rate', 0.1, 'purchase_price', 18, 'inspection_cost', 3), ...
        'Product', struct('defect_rate', 0.1, 'assembly_cost', 6, 'inspection_cost', 3)), ...
    'Scenario2', struct('Part1', struct('defect_rate', 0.2, 'purchase_price', 4, 'inspection_cost',
2), ...
        'Part2', struct('defect_rate', 0.2, 'purchase_price', 18, 'inspection_cost', 3), ...
        'Product', struct('defect_rate', 0.2, 'assembly_cost', 6, 'inspection_cost', 3)), ...
    'Scenario3', struct('Part1', struct('defect_rate', 0.1, 'purchase_price', 4, 'inspection_cost',
2), ...
        'Part2', struct('defect_rate', 0.1, 'purchase_price', 18, 'inspection_cost', 3), ...
        'Product', struct('defect_rate', 0.1, 'assembly_cost', 6, 'inspection_cost', 3))), ...

```

```

'Scenario4', struct('Part1', struct('defect_rate', 0.2, 'purchase_price', 4, 'inspection_cost',
1), ...
    'Part2', struct('defect_rate', 0.2, 'purchase_price', 18, 'inspection_cost', 1), ...
    'Product', struct('defect_rate', 0.2, 'assembly_cost', 6, 'inspection_cost', 2)), ...
'Scenario5', struct('Part1', struct('defect_rate', 0.1, 'purchase_price', 4, 'inspection_cost',
8), ...
    'Part2', struct('defect_rate', 0.2, 'purchase_price', 18, 'inspection_cost', 1), ...
    'Product', struct('defect_rate', 0.1, 'assembly_cost', 6, 'inspection_cost', 2)), ...
'Scenario6', struct('Part1', struct('defect_rate', 0.05, 'purchase_price', 4, 'inspection_cost',
2), ...
    'Part2', struct('defect_rate', 0.05, 'purchase_price', 18, 'inspection_cost', 3), ...
    'Product', struct('defect_rate', 0.05, 'assembly_cost', 6, 'inspection_cost', 3)) ...
);

% 定义采样数据
sample_data = struct(...
    'Part1', struct('k', 5, 'n', 50), ...
    'Part2', struct('k', 10, 'n', 50), ...
    'Product', struct('k', 8, 'n', 50) ...
);

% 初始化先验参数
alpha_pro = 1;
beta_pro = 1;

% 初始化结果数组
update_results = cell(length(fieldnames(ifs)), 1);

%% 调用函数，计算具体的值
% 遍历每个场景
if_names = fieldnames(ifs);
for i = 1:length(if_names)
    scenario_name = if_names{i};
    scenario_data = ifs.(scenario_name);

    result = struct('Scenario', scenario_name);

    % 更新 Part1 的次品率
    k1 = sample_data.Part1.k;
    n1 = sample_data.Part1.n;
    [alpha_post1, beta_post1] = beiyesi_update(k1, n1, alpha_pro, beta_pro);
    updated_theta1 = expected_defective_rate(alpha_post1, beta_post1);
    result.Part1_updated_defect_rate = updated_theta1;

    % 更新 Part2 的次品率
    k2 = sample_data.Part2.k;
    n2 = sample_data.Part2.n;
    [alpha_post2, beta_post2] = beiyesi_update(k2, n2, alpha_pro, beta_pro);
    updated_theta2 = expected_defective_rate(alpha_post2, beta_post2);

```

```

result.Part2_updated_defect_rate = updated_theta2;

% 更新 Product 的次品率
k_prod = sample_data.Product.k;
n_prod = sample_data.Product.n;
[alpha_post_prod, beta_post_prod] = beiyesi_update(k_prod, n_prod, alpha_pro,
beta_pro);
update_theta_prod = expected_defective_rate(alpha_post_prod, beta_post_prod);
result.Product_updated_defect_rate = update_theta_prod;

% 将结果存储到数组中
update_results{i} = result;
end

% 将结果转换为表格并打印
results_table = struct2table(cell2mat(update_results));
disp(results_table);

%% 函数定义
% 定义贝叶斯更新函数
function [alpha_post, beta_post] = beiyesi_update(k, n, alpha_prior, beta_pro)
    alpha_post = alpha_prior + k;
    beta_post = beta_pro + (n - k);
end

% 计算后验概率
function theta = expected_defective_rate(alpha_after, beta_after)
    theta = alpha_after / (alpha_after + beta_after);
end

```

问题四

问题二更新求解(MATBAL)

```

%% 代码说明
% 求解第四问-问题二情况下的代码
% 训练鲸鱼算法
% 鲸鱼数量: 10
% 最大迭代数为: 50
% 决策变量的维度为: 4

%% 主程序
clc
clear
jingyushu = 10; % 鲸鱼数量
MAXGEN = 50; % 最大迭代次数
weidu = 4; % 决策变量的维度
ub = 1 * ones(1, weidu); % 决策变量的上界

```

```

lb = 0 * ones(1, weidu); % 决策变量的下界
L_p = zeros(1, weidu); % 最优解的位置
id = 6; % 情况编号
L_s = inf; % 最优解的适应度值
PS = rand(jingyushu, weidu) .* repmat(ub - lb, jingyushu, 1) + repmat(lb, jingyushu, 1); %
初始化鲸鱼位置
trace = zeros(1, MAXGEN); % 记录每一代的最优适应度值

% 定义 n1 和 n2 的范围
n1_values = 100:100:1000;
n2_values = 100:100:1000;

% 初始化结果表格
results = cell(length(n1_values) * length(n2_values), 8);
row_index = 1;

for n1 = n1_values
    for n2 = n1
        % 重置最优解
        L_p = zeros(1, weidu);
        L_s = inf;

        for kk = 1:MAXGEN
            % 更新参数
            a = 2 - kk * (2 / MAXGEN); % 线性递减的参数 a
            a2 = -1 + kk * (1 / MAXGEN); % 线性递减的参数 a2

            % 计算适应度值并更新最优解
            for i = 1:jingyushu
                % 边界处理：越界赋值为边界值
                PS(i, :) = min(max(PS(i, :), lb), ub);

                % 计算适应度值
                fitness = ObjFun(PS(i, :), id, n1, n2);

                % 更新最优解
                if fitness < L_s
                    L_s = fitness;
                    L_p = PS(i, :);
                end
            end

            % 更新鲸鱼位置
            r1 = rand(jingyushu, 1);
            r2 = rand(jingyushu, 1);
            A = 2 * a * r1 - a; % 参数 A
            C = 2 * r2; % 参数 C

```

```

b = 1;          % 螺旋参数
l = (a2 - 1) * rand(jingyushu, 1) + 1; % 螺旋参数
p = rand(jingyushu, 1); % 随机选择更新方式

for i = 1:jingyushu
    if p(i) < 0.5 % 包围捕食
        if abs(A(i)) < 1
            D = abs(C(i) * L_p - PS(i, :));
            PS(i, :) = L_p - A(i) * D;
        else
            rand_leader_index = randi(jingyushu);
            X_rand = PS(rand_leader_index, :);
            D_X_rand = abs(C(i) * X_rand - PS(i, :));
            PS(i, :) = X_rand - A(i) * D_X_rand;
        end
    else % 螺旋更新
        distance_LLeader = abs(L_p - PS(i, :));
        PS(i, :) = distance_LLeader .* exp(b * l(i)) .* cos(l(i) * 2 * pi) + L_p;
    end
end

% 记录当前代的最优适应度值
trace(kk) = L_s;
end

% 结果输出
[ans1, ans2] = ObjFun1(L_p, id, n1, n2);

% 保存结果到表格
results{row_index, 1} = n1;
results{row_index, 2} = n2;
results{row_index, 3} = L_p(1);
results{row_index, 4} = L_p(2);
results{row_index, 5} = L_p(3);
results{row_index, 6} = L_p(4);
results{row_index, 7} = ans1;
results{row_index, 8} = ans2 * 56;

row_index = row_index + 1;
end
end

% 显示结果表格
disp("第"+id+"种情况下的决策方案");
results_table = cell2table(results, 'VariableNames', {'n1', 'n2', '零配件 1 是否检测', '零配件 2 是否检测', '成品是否检测', '不合格成品是否拆解', '成本', '收入'});
disp(results_table);

```

%% 模拟题目中的步骤 1、2

```
function [money1, inco] = ObjFun1(ttt, id, n1, n2)
    maa = [0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 30 5;
           0.115385 4 1 0.211538 18 1 0.173077 6 2 56 30 5;
           0.115385 4 8 0.211538 18 1 0.173077 6 2 56 10 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 10 40];
    a = maa(id, 1); %零部件 1 的次品率
    b = maa(id, 2); %零部件 1 的购买单价
    c = maa(id, 3); %零部件 1 的检测成本
    a1 = maa(id, 4); %零部件 2 的次品率
    b1 = maa(id, 5); %零部件 2 的购买单价
    c1 = maa(id, 6); %零部件 2 的检测成本
    a2 = maa(id, 7); %成品的次品率
    b2 = maa(id, 8); %成品的装配成本
    c2 = maa(id, 9); %成品的检测成本
    d2 = maa(id, 10); %成品的市场售价
    a3 = maa(id, 11); %不合格成品的调换损失
    b3 = maa(id, 12); %不合格成品的拆解费用
    ce1 = ttt(1);
    ce2 = ttt(2);
    ce3 = ttt(3);
    ce4 = ttt(4);
    theta1 = min(n1 * ce1 * (1 - a) + n1 * (1 - ce1), n2 * ce2 * (1 - a1) + n2 * (1 - ce2));
    beta = a + a1 + a2 - a * a1 - a * a2 - a1 * a2 + a * a1 * a2;
    inco = theta1 * ce3 * (1 - beta) + theta1 * (1 - ce3) * beta;
    money1 = n1 * b + n2 * b1 + n1 * ce1 * c + n2 * ce2 * c1 + theta1 * b2 + theta1 * ce3 *
    c2 + theta1 * (1 - ce3) * a3 * beta + theta1 * beta * ce4 * b3;
    nn = theta1 * beta * ce4;
    theta2 = min(nn * ce1 * (1 - a) + nn * (1 - ce1), nn * ce2 * (1 - a1) + nn * (1 - ce2));
    money1 = money1 + nn * (ce1 * c + ce2 * c) + theta2 * b2 + theta2 * ce3 * c2;
    inco = inco + theta2 * ce3 * (1 - beta) + theta2 * (1 - ce3) * beta;
end
```

%% 模拟题目中的步骤 3、4

```
function revenue = ObjFun(ttt, id, n1, n2)
    maa = [0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 30 5;
           0.115385 4 1 0.211538 18 1 0.173077 6 2 56 30 5;
           0.115385 4 8 0.211538 18 1 0.173077 6 2 56 10 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 10 40];
    a = maa(id, 1); %零部件 1 的次品率
    b = maa(id, 2); %零部件 1 的购买单价
    c = maa(id, 3); %零部件 1 的检测成本
    a1 = maa(id, 4); %零部件 2 的次品率
```

```

b1 = maa(id, 5); %零部件 2 的购买单价
c1 = maa(id, 6); %零部件 2 的检测成本
a2 = maa(id, 7); %成品的次品率
b2 = maa(id, 8); %成品的装配成本
c2 = maa(id, 9); %成品的检测成本
d2 = maa(id, 10); %成品的市场售价
a3 = maa(id, 11); %不合格成品的调换损失
b3 = maa(id, 12); %不合格成品的拆解费用
ce1 = ttt(1);
ce2 = ttt(2);
ce3 = ttt(3);
ce4 = ttt(4);
theta1 = min(n1 * ce1 * (1 - a) + n1 * (1 - ce1), n2 * ce2 * (1 - a1) + n2 * (1 - ce2));
beta = a + a1 + a2 - a * a1 - a * a2 - a1 * a2 + a * a1 * a2;
inco = theta1 * ce3 * (1 - beta) + theta1 * (1 - ce3) * beta;
money2 = n1 * b + n2 * b1 + n1 * ce1 * c + n2 * ce2 * c1 + theta1 * b2 + theta1 * ce3 *
c2 + theta1 * (1 - ce3) * a3 * beta + theta1 * beta * ce4 * b3;
nn = theta1 * beta * ce4;
theta2 = min(nn * ce1 * (1 - a) + nn * (1 - ce1), nn * ce2 * (1 - a1) + nn * (1 - ce2));
money2 = money2 + nn * (ce1 * c + ce2 * c) + theta2 * b2 + theta2 * ce3 * c2;
inco = inco + theta2 * ce3 * (1 - beta) + theta2 * (1 - ce3) * beta;
revenue = money2 - inco * 56;
end

```

问题四

问题二训练鲸鱼算法(MATBAL)

```

%% 代码说明
%用训练好的鲸鱼算法进行求解第四问_问题二
%通过修改第 17 行的 id, 可以得到六种情况的决策方案
%id 的值与题目中的 1、2、3、4、5、6 相对应

%% 主程序
clc
clear
jingyushu = 10; % 鲸鱼数量
MAXGEN = 50; % 最大迭代次数
weidu = 4; % 决策变量的维度
ub = 1 * ones(1, weidu); % 决策变量的上界
lb = 0 * ones(1, weidu); % 决策变量的下界
L_p = zeros(1, weidu); % 最优解的位置
%修改 id, 即可得到六种情况下的决策方案
id = 1; % 情况编号
L_s = inf; % 最优解的适应度值
PS = rand(jingyushu, weidu) .* repmat(ub - lb, jingyushu, 1) + repmat(lb, jingyushu, 1); %
初始化鲸鱼位置
trace = zeros(1, MAXGEN); % 记录每一代的最优适应度值

```

```

for kk = 1:MAXGEN
    % 更新参数
    a = 2 - kk * (2 / MAXGEN); % 线性递减的参数 a
    a2 = -1 + kk * (1 / MAXGEN); % 线性递减的参数 a2

    for i = 1:jingyushu
        % 边界处理：越界赋值为边界值
        ubb = PS(i, :) > ub;
        lbb = PS(i, :) < lb;
        PS(i, :) = (PS(i, :).*(~(ubb + lbb))) + ub.*ubb + lb.*lbb;

        % 计算适应度值
        fitness = ObjFun(PS(i, :), id);

        % 更新最优解
        if fitness < L_s
            L_s = fitness;
            L_p = PS(i, :);
        end
    end

    % 更新鲸鱼位置
    for i = 1:jingyushu
        r1 = rand(); r2 = rand();
        A = 2 * a * r1 - a; % 参数 A
        C = 2 * r2; % 参数 C
        b = 1; % 螺旋参数
        l = (a2 - 1) * rand() + 1; % 螺旋参数
        p = rand(); % 随机选择更新方式

        for j = 1:weidu
            if p < 0.5 % 包围捕食
                if abs(A) < 1
                    D = abs(C * L_p(j) - PS(i, j));
                    PS(i, j) = L_p(j) - A * D;
                else
                    rand_leader_index = floor(jingyushu * rand() + 1);
                    X_rand = PS(rand_leader_index, :);
                    D_X_rand = abs(C * X_rand(j) - PS(i, j));
                    PS(i, j) = X_rand(j) - A * D_X_rand;
                end
            else % 螺旋更新
                distance_LLeader = abs(L_p(j) - PS(i, j));
                PS(i, j) = distance_LLeader * exp(b * l) * cos(l * 2 * pi) + L_p(j);
            end
        end
    end
end

```

```

% 记录当前代的最优适应度值
trace(kk) = L_s;
end

%% 结果输出
name = ["零配件 1 是否检测", "零配件 2 是否检测", "成品是否检测", "不合格成品是否
拆解"];
[ans1, ans2] = ObjFun1(L_p, id);
plot(trace, 'LineWidth', 2);
xlabel('迭代次数');
ylabel('适应度值');
title('适应度曲线');
fprintf("情况: %d\n", id);
answer = [name; string(L_p)];
for i = 1:4
    disp(name(i) + ':' + num2str(L_p(i)));
end
disp('成本:');
disp(ans1);
disp('收入:');
disp(ans2 * 56);

%% 目标函数
function anss = ObjFun(ttt, id)
    n1 = 100; n2 = 100;
    maa = [0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 30 5;
           0.115385 4 1 0.211538 18 1 0.173077 6 2 56 30 5;
           0.115385 4 8 0.211538 18 1 0.173077 6 2 56 10 5;
           0.115385 4 2 0.211538 18 3 0.173077 6 3 56 10 40];
    a = maa(id, 1); b = maa(id, 2); c = maa(id, 3);
    a1 = maa(id, 4); b1 = maa(id, 5); c1 = maa(id, 6);
    a2 = maa(id, 7); b2 = maa(id, 8); c2 = maa(id, 9); d2 = maa(id, 10);
    a3 = maa(id, 11); b3 = maa(id, 12);
    ce1 = ttt(1); ce2 = ttt(2); ce3 = ttt(3); ce4 = ttt(4);
    D = min(n1 * ce1 * (1 - a) + n1 * (1 - ce1), n2 * ce2 * (1 - a1) + n2 * (1 - ce2));
    beta = a + a1 + a2 - a * a1 - a * a2 - a1 * a2 + a * a1 * a2;
    inco = D * ce3 * (1 - beta) + D * (1 - ce3) * beta;
    mon = n1 * b + n2 * b1 + n1 * ce1 * c + n2 * ce2 * c1 + D * b2 + D * ce3 * c2 + D * (1 -
ce3) * a3 * beta + D * beta * ce4 * b3;
    nn = D * beta * ce4;
    DD = min(nn * ce1 * (1 - a) + nn * (1 - ce1), nn * ce2 * (1 - a1) + nn * (1 - ce2));
    mon = mon + nn * (ce1 * c + ce2 * c) + DD * b2 + DD * ce3 * c2;
    inco = inco + DD * ce3 * (1 - beta) + DD * (1 - ce3) * beta;
    anss = mon - inco * 56;
end

```

```

%% 目标函数 1
function [mon, inco] = ObjFun1(ttt, id)
    n1 = 100; n2 = 100;
    maa = [0.15 4 2 0.15 18 3 0.15 6 3 56 6 5;
           0.15 4 2 0.15 18 3 0.14 6 3 56 6 5;
           0.15 4 2 0.15 18 3 0.15 6 3 56 30 5;
           0.15 4 1 0.15 18 1 0.15 6 2 56 30 5;
           0.15 4 8 0.15 18 1 0.15 6 2 56 10 5;
           0.15 4 2 0.15 18 3 0.14 6 3 56 10 40];
    a = maa(id, 1); b = maa(id, 2); c = maa(id, 3);
    a1 = maa(id, 4); b1 = maa(id, 5); c1 = maa(id, 6);
    a2 = maa(id, 7); b2 = maa(id, 8); c2 = maa(id, 9); d2 = maa(id, 10);
    a3 = maa(id, 11); b3 = maa(id, 12);
    ce1 = ttt(1); ce2 = ttt(2); ce3 = ttt(3); ce4 = ttt(4);
    D = min(n1 * ce1 * (1 - a) + n1 * (1 - ce1), n2 * ce2 * (1 - a1) + n2 * (1 - ce2));
    beta = a + a1 + a2 - a * a1 - a * a2 - a1 * a2 + a * a1 * a2;
    inco = D * ce3 * (1 - beta) + D * (1 - ce3) * beta;
    mon = n1 * b + n2 * b1 + n1 * ce1 * c + n2 * ce2 * c1 + D * b2 + D * ce3 * c2 + D * (1 - ce3) * a3 * beta + D * beta * ce4 * b3;
    nn = D * beta * ce4;
    DD = min(nn * ce1 * (1 - a) + nn * (1 - ce1), nn * ce2 * (1 - a1) + nn * (1 - ce2));
    mon = mon + nn * (ce1 * c + ce2 * c) + DD * b2 + DD * ce3 * c2;
    inco = inco + DD * ce3 * (1 - beta) + DD * (1 - ce3) * beta;
end

```

问题四

更新问题二检验(MATBAL)

```

%% 主程序
clc
clear
jingyushu_values = [10, 20, 30];
MAXGEN_values = [50, 100, 150];
weidu_values = [2, 4, 6];

% 初始化结果表格
results = cell(length(jingyushu_values) * length(MAXGEN_values) * length(weidu_values), 10);
row_index = 1;

for jingyushu = jingyushu_values
    for MAXGEN = MAXGEN_values
        for weidu = weidu_values
            % 初始化参数
            ub = 1 * ones(1, weidu); % 决策变量的上界
            lb = 0 * ones(1, weidu); % 决策变量的下界
            L_p = zeros(1, weidu); % 最优解的位置

```

```

id = 2;          % 情况编号
L_s = inf;      % 最优解的适应度值
PS = rand(jingyushu, weidu) .* repmat(ub - lb, jingyushu, 1) + repmat(lb, jingyushu, 1); % 初始化鲸鱼位置
trace = zeros(1, MAXGEN); % 记录每一代的最优适应度值

% 定义 n1 和 n2 的范围
n1_values = 100:100:1000;
n2_values = 100:100:1000;

for n1 = n1_values
    for n2 = n1
        % 重置最优解
        L_p = zeros(1, weidu);
        L_s = inf;

        for kk = 1:MAXGEN
            % 更新参数
            a = 2 - kk * (2 / MAXGEN); % 线性递减的参数a
            a2 = -1 + kk * (1 / MAXGEN); % 线性递减的参数a2

            % 计算适应度值并更新最优解
            for i = 1:jingyushu
                % 边界处理：越界赋值为边界值
                PS(i, :) = min(max(PS(i, :), lb), ub);

                % 计算适应度值
                fitness = ObjFun(PS(i, :), id, n1, n2);

                % 更新最优解
                if fitness < L_s
                    L_s = fitness;
                    L_p = PS(i, :);
                end
            end

            % 更新鲸鱼位置
            r1 = rand(jingyushu, 1);
            r2 = rand(jingyushu, 1);
            A = 2 * a * r1 - a; % 参数A
            C = 2 * r2;      % 参数C
            b = 1;          % 螺旋参数
            l = (a2 - 1) * rand(jingyushu, 1) + 1; % 螺旋参数
            p = rand(jingyushu, 1); % 随机选择更新方式

            for i = 1:jingyushu
                if p(i) < 0.5 % 包围捕食

```

```

        if abs(A(i)) < 1
            D = abs(C(i) * L_p - PS(i, :));
            PS(i, :) = L_p - A(i) * D;
        else
            rand_leader_index = randi(jingyushu);
            X_rand = PS(rand_leader_index, :);
            D_X_rand = abs(C(i) * X_rand - PS(i, :));
            PS(i, :) = X_rand - A(i) * D_X_rand;
        end
    else % 螺旋更新
        distance_Loader = abs(L_p - PS(i, :));
        PS(i, :) = distance_Loader .* exp(b * l(i)) .* cos(l(i) * 2 * pi) + L_p;
    end
end

% 记录当前代的最优适应度值
trace(kk) = L_s;
end

% 结果输出
[ans1, ans2] = ObjFun1(L_p, id, n1, n2);

% 保存结果到表格
results{row_index, 1} = jingyushu;
results{row_index, 2} = MAXGEN;
results{row_index, 3} = weidu;
results{row_index, 4} = n1;
results{row_index, 5} = n2;
results{row_index, 6} = L_p(1);
results{row_index, 7} = L_p(2);
if length(L_p) >= 3
    results{row_index, 8} = L_p(3);
else
    results{row_index, 8} = NaN;
end
if length(L_p) >= 4
    results{row_index, 9} = L_p(4);
else
    results{row_index, 9} = NaN;
end
results{row_index, 10} = L_s;

row_index = row_index + 1;
end
end
end
end
end

```

```

% 显示结果表格
results_table = cell2table(results, 'VariableNames', {'jingyushu', 'MAXGEN', 'weidu', 'n1',
'n2', '零配件1是否检测', '零配件2是否检测', '成品是否检测', '不合格成品是否拆解', '最
优适应度值'});
disp(results_table);

% 创建文件名
filename = 't4_test_results.xlsx';
% 将结果表写入Excel文件
writetable(results_table, filename);

%% 可视化处理
% 绘制不同参数设置下的最优适应度值
figure;
hold on;
for i = 1:length(jingyushu_values)
    for j = 1:length(MAXGEN_values)
        for k = 1:length(weidu_values)
            idx = (i-1)*length(MAXGEN_values)*length(weidu_values) + (j-
1)*length(weidu_values) + k;
            plot(results{idx, 10}, 'DisplayName', sprintf('jingyushu=%d, MAXGEN=%d,
weidu=%d', results{idx, 1}, results{idx, 2}, results{idx, 3}));
        end
    end
end

%% 目标函数
function anss = ObjFun(ttt, id, n1, n2)
    maa = [0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
0.115385 4 2 0.211538 18 3 0.173077 6 3 56 6 5;
0.115385 4 2 0.211538 18 3 0.173077 6 3 56 30 5;
0.115385 4 1 0.211538 18 1 0.173077 6 2 56 30 5;
0.115385 4 8 0.211538 18 1 0.173077 6 2 56 10 5;
0.115385 4 2 0.211538 18 3 0.173077 6 3 56 10 40];
    a = maa(id, 1); b = maa(id, 2); c = maa(id, 3);
    a1 = maa(id, 4); b1 = maa(id, 5); c1 = maa(id, 6);
    a2 = maa(id, 7); b2 = maa(id, 8); c2 = maa(id, 9); d2 = maa(id, 10);
    a3 = maa(id, 11); b3 = maa(id, 12);

% 检查维度并设置默认值
ce1 = ttt(1);
ce2 = ttt(2);
ce3 = 0;
ce4 = 0;
if length(ttt) >= 3
    ce3 = ttt(3);
end

```

```

if length(ttt) >= 4
    ce4 = ttt(4);
end

D = min(n1 * ce1 * (1 - a) + n1 * (1 - ce1), n2 * ce2 * (1 - a1) + n2 * (1 - ce2));
beta = a + a1 + a2 - a * a1 - a * a2 - a1 * a2 + a * a1 * a2;
inco = D * ce3 * (1 - beta) + D * (1 - ce3) * beta;
mon = n1 * b + n2 * b1 + n1 * ce1 * c + n2 * ce2 * c1 + D * b2 + D * ce3 * c2 + D * (1 -
ce3) * a3 * beta + D * beta * ce4 * b3;
nn = D * beta * ce4;
DD = min(nn * ce1 * (1 - a) + nn * (1 - ce1), nn * ce2 * (1 - a1) + nn * (1 - ce2));
mon = mon + nn * (ce1 * c + ce2 * c) + DD * b2 + DD * ce3 * c2;
inco = inco + DD * ce3 * (1 - beta) + DD * (1 - ce3) * beta;
anss = mon - inco * 56;
end

%% 目标函数1
function [mon, inco] = ObjFun1(ttt, id, n1, n2)
    maa = [0.15 4 2 0.15 18 3 0.15 6 3 56 6 5;
            0.15 4 2 0.15 18 3 0.14 6 3 56 6 5;
            0.15 4 2 0.15 18 3 0.15 6 3 56 30 5;
            0.15 4 1 0.15 18 1 0.15 6 2 56 30 5;
            0.15 4 8 0.15 18 1 0.15 6 2 56 10 5;
            0.15 4 2 0.15 18 3 0.14 6 3 56 10 40];
    a = maa(id, 1); b = maa(id, 2); c = maa(id, 3);
    a1 = maa(id, 4); b1 = maa(id, 5); c1 = maa(id, 6);
    a2 = maa(id, 7); b2 = maa(id, 8); c2 = maa(id, 9); d2 = maa(id, 10);
    a3 = maa(id, 11); b3 = maa(id, 12);

    % 检查维度并设置默认值
    ce1 = ttt(1);
    ce2 = ttt(2);
    ce3 = 0;
    ce4 = 0;
    if length(ttt) >= 3
        ce3 = ttt(3);
    end
    if length(ttt) >= 4
        ce4 = ttt(4);
    end

    D = min(n1 * ce1 * (1 - a) + n1 * (1 - ce1), n2 * ce2 * (1 - a1) + n2 * (1 - ce2));
    beta = a + a1 + a2 - a * a1 - a * a2 - a1 * a2 + a * a1 * a2;
    inco = D * ce3 * (1 - beta) + D * (1 - ce3) * beta;
    mon = n1 * b + n2 * b1 + n1 * ce1 * c + n2 * ce2 * c1 + D * b2 + D * ce3 * c2 + D * (1 -
ce3) * a3 * beta + D * beta * ce4 * b3;
    nn = D * beta * ce4;
    DD = min(nn * ce1 * (1 - a) + nn * (1 - ce1), nn * ce2 * (1 - a1) + nn * (1 - ce2));
    mon = mon + nn * (ce1 * c + ce2 * c) + DD * b2 + DD * ce3 * c2;

```

```

inco = inco + DD * ce3 * (1 - beta) + DD * (1 - ce3) * beta;
end

```

问题四

问题三贝叶斯更新(MATBAL)

%% 代码说明

%通过贝叶斯更新函数，在问题二的背景下，该代码计算零配件1、配件2和成品的后验概率

%% 数据初始化

clc

clear

%初始化零配件的数据

%defect_rate为次品率 purchase_price为购买单价 inspection_cost为检测成本

components = struct(...

'Part1', struct('defect_rate', 0.1, 'purchase_price', 2, 'inspection_cost', 1), ...

'Part2', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...

'Part3', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2), ...

'Part4', struct('defect_rate', 0.1, 'purchase_price', 2, 'inspection_cost', 1), ...

'Part5', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...

'Part6', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2), ...

'Part7', struct('defect_rate', 0.1, 'purchase_price', 8, 'inspection_cost', 1), ...

'Part8', struct('defect_rate', 0.1, 'purchase_price', 12, 'inspection_cost', 2) ...

);

% 初始化半成品数据

pro_products = struct(...

'SemiProduct1', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4, 'disassembly_cost', 6), ...

'SemiProduct2', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4, 'disassembly_cost', 6), ...

'SemiProduct3', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 4, 'disassembly_cost', 6) ...

);

% 初始化成品数据

final_product = struct(...

'FinalProduct', struct('defect_rate', 0.1, 'assembly_cost', 8, 'inspection_cost', 6, 'disassembly_cost', 10, 'selling_price', 200, 'replacement_loss', 40) ...

);

% 初始化采样数据

sample_data = struct(...

```

'Part1', struct('k', 5, 'n', 50), ...
'Part2', struct('k', 10, 'n', 50), ...
'Part3', struct('k', 8, 'n', 50), ...
'Part4', struct('k', 3, 'n', 50), ...
'Part5', struct('k', 7, 'n', 50), ...
'Part6', struct('k', 2, 'n', 50), ...
'Part7', struct('k', 6, 'n', 50), ...
'Part8', struct('k', 4, 'n', 50), ...
'SemiProduct1', struct('k', 6, 'n', 50), ...
'SemiProduct2', struct('k', 9, 'n', 50), ...
'SemiProduct3', struct('k', 5, 'n', 50), ...
'FinalProduct', struct('k', 7, 'n', 50) ...
);

% 初始化先验参数
alpha_pro = 1;
beta_pro = 1;

% 初始化结果数组
update_results = cell(length(fieldnames(components)) + length(fieldnames(pro_products)) +
length(fieldnames(final_product)), 1);

% 合并组件、半成品和成品数据
all_items = [fieldnames(components); fieldnames(pro_products);
fieldnames(final_product)];

%% 调用函数，计算具体的值
% 遍历每个项目
for i = 1:length(all_items)
    if_name = all_items{i};
    k = sample_data.(if_name).k;
    n = sample_data.(if_name).n;

    [alpha_post, beta_post] = beiyesi_update(k, n, alpha_pro, beta_pro);
    updated_theta = expected_defective_rate(alpha_post, beta_post);

    result = struct('Item', if_name, 'Updated_Defect_Rate', updated_theta);
    updated_results{i} = result;
end

% 将结果转换为表格并打印
results_table = struct2table(cell2mat(updated_results));
disp(results_table);

%% 函数定义
% 定义贝叶斯更新函数
function [alpha_post, beta_post] = beiyesi_update(k, n, alpha_prior, beta_pro)
    alpha_post = alpha_prior + k;
    beta_post = beta_pro + (n - k);

```

```
end
```

```
% 计算后验概率
```

```
function theta = expected_defective_rate(alpha_after, beta_after)
```

```
    theta = alpha_after / (alpha_after + beta_after);
```

```
end
```

问题四

问题三鲸鱼算法训练(MATBAL)

```
%% 代码说明
```

```
%用训练好的鲸鱼算法进行求解第四问_问题三
```

```
clc
```

```
clear
```

```
%% 主程序
```

```
jingyushu = 10; % 鲸鱼数量
```

```
MAXGEN = 100; % 最大迭代次数
```

```
weidu = 16; % 决策变量的维度
```

```
ub = 1 * ones(1, weidu); % 决策变量的上界
```

```
lb = 0 * ones(1, weidu); % 决策变量的下界
```

```
L_p = zeros(1, weidu); % 最优解的位置
```

```
id = 1; % 情况编号
```

```
L_s = inf; % 最优解的适应度值
```

```
PS = rand(jingyushu, weidu) .* repmat(ub - lb, jingyushu, 1) + repmat(lb, jingyushu, 1); %
```

```
初始化鲸鱼位置
```

```
trace = zeros(1, MAXGEN); % 记录每一代的最优适应度值
```

```
for kk = 1:MAXGEN
```

```
    % 更新参数
```

```
    a = 2 - kk * (2 / MAXGEN); % 线性递减的参数a
```

```
    a2 = -1 + kk * (1 / MAXGEN); % 线性递减的参数a2
```

```
    for i = 1:jingyushu
```

```
        % 边界处理：越界赋值为边界值
```

```
        ubb = PS(i, :) > ub;
```

```
        lbb = PS(i, :) < lb;
```

```
        PS(i, :) = (PS(i, :).*(~(ubb + lbb))) + ub.*ubb + lb.*lbb;
```

```
    % 计算适应度值
```

```
    fitness = ObjFun(PS(i, :), id);
```

```
    % 更新最优解
```

```
    if fitness < L_s
```

```
        L_s = fitness;
```

```
        L_p = PS(i, :);
```

```

end
end

% 更新鲸鱼位置
for i = 1:jingyushu
    r1 = rand(); r2 = rand();
    A = 2 * a * r1 - a; % 参数A
    C = 2 * r2; % 参数C
    b = 1; % 螺旋参数
    l = (a2 - 1) * rand() + 1; % 螺旋参数
    p = rand(); % 随机选择更新方式

    for j = 1:weidu
        if p < 0.5 % 包围捕食
            if abs(A) < 1
                D = abs(C * L_p(j) - PS(i, j));
                PS(i, j) = L_p(j) - A * D;
            else
                rand_leader_index = floor(jingyushu * rand() + 1);
                X_rand = PS(rand_leader_index, :);
                D_X_rand = abs(C * X_rand(j) - PS(i, j));
                PS(i, j) = X_rand(j) - A * D_X_rand;
            end
        else % 螺旋更新
            distance_LLeader = abs(L_p(j) - PS(i, j));
            PS(i, j) = distance_LLeader * exp(b * l) * cos(l * 2 * pi) + L_p(j);
        end
    end
end

% 记录当前代的最优适应度值
trace(kk) = L_s;
end

%% 结果输出
name = ["零件1检测率", "零件2检测率", "零件3检测率", "零件4检测率", "零件5检测率",
"零件6检测率", "零件7检测率", "零件8检测率", "半成品1检测率", "半成品2检测率",
"半成品3检测率", "半成品1拆解率", "半成品2拆解率", "半成品3拆解率", "成品检测率",
"成品拆解率"];
[ans1, ans2] = ObjFun1(L_p, id);
plot(trace, 'LineWidth', 2);
xlabel('迭代次数');
ylabel('适应度值');
title('适应度曲线');
answer = [name; string(L_p)];
for i = 1:16
    disp(name(i) + ':' + num2str(L_p(i)));
end

```

```

end
disp('成本:');
disp(ans1);
disp('收入:');
disp(ans2 * 200);

%% 目标函数
function anss = ObjFun(ttt, id)
    n = [100 100 100 100 100 100 100 100];
    n = n * 1; inco = 0;
    mon = 0;
    ma1 = [0.11538 2 1; 0.21154 8 1; 0.17308 12 2; 0.076923 2 1; 0.15385 8 1; 0.057692 12
2; 0.13462 8 1; 0.096154 12 2];
    ma2 = [0.13462 8 4 6; 0.19231 8 4 6; 0.11538 8 4 6];
    ma3 = [0.15385 8 6 10];
    for zzz = 1:2
        H = [1000 1000 1000];
        D = 1000;
        beta = 0;
        for i = 1:8
            mon = mon + ma1(i, 2) * ma1(i, 3) + ttt(i) * ma1(i, 1) * ma1(i, 3);
        end
        for i = 1:3
            H(1) = min(n(i) * ma1(i, 1) * (1 - ttt(i)) + n(i) * (1 - ma1(i, 1)), H(1));
        end
        for i = 1:3
            H(2) = min(n(i + 3) * ma1(i + 3, 1) * (1 - ttt(i)) + n(i + 3) * (1 - ma1(i + 3, 1)), H(2));
        end
        for i = 1:2
            H(3) = min(n(i + 6) * ma1(i + 6, 1) * (1 - ttt(i)) + n(i + 6) * (1 - ma1(i + 6, 1)), H(3));
        end
        for i = 1:3
            mon = mon + H(i) * ttt(i + 8) * ma2(i, 3) + H(i) * ma2(i, 2) + H(i) * ttt(i + 8) * ma2(i,
1) * ma2(i, 4) * ttt(i + 11);
        end
        for i = 1:3
            D = min(H(i) * ma2(i, 1) * (1 - ttt(i + 8)) + H(i) * (1 - ma2(i, 1)), D);
        end
        z = [ma2(1, 1) ma2(2, 1) ma2(3, 1) ma3(1)];
        for i = 1:4
            beta = beta + z(i);
            for j = i:4
                beta = beta - z(i) * z(j);
                for k = j:4
                    beta = beta + z(i) * z(j) * z(k);
                    for kk = k:4
                        beta = beta - z(i) * z(j) * z(k) * z(kk);
                    end
                end
            end
        end
    end
end

```

```

        end
    end
    mon = mon + D * ma3(2) + D * ttt(14) * ma3(3) + D * (1 - ttt(14)) * 40 * beta + D *
beta * ttt(15) * 40;
    nn = D * beta * ttt(15);
    n = nn * n / 100;
    inco = inco + D * ttt(14) * (1 - beta) + D * (1 - ttt(14)) * beta;
    end
    anss = mon - inco * 200;
end

%% 目标函数1
function [mon, inco] = ObjFun1(ttt, id)
    n = [100 100 100 100 100 100 100 100];
    n = n * 1; inco = 0;
    mon = 0;
    ma1 = [0.11538 2 1; 0.21154 8 1; 0.17308 12 2; 0.076923 2 1; 0.15385 8 1; 0.057692 12
2; 0.13462 8 1; 0.096154 12 2];
    ma2 = [0.13462 8 4 6; 0.19231 8 4 6; 0.11538 8 4 6];
    ma3 = [0.15385 8 6 10];
    for zzz = 1:2
        H = [1000 1000 1000];
        D = 1000;
        beta = 0;
        for i = 1:8
            mon = mon + ma1(i, 2) * ma1(i, 3) + ttt(i) * ma1(i, 1) * ma1(i, 3);
        end
        for i = 1:3
            H(1) = min(n(i) * ma1(i, 1) * (1 - ttt(i)) + n(i) * (1 - ma1(i, 1)), H(1));
        end
        for i = 1:3
            H(2) = min(n(i + 3) * ma1(i + 3, 1) * (1 - ttt(i)) + n(i + 3) * (1 - ma1(i + 3, 1)), H(2));
        end
        for i = 1:2
            H(3) = min(n(i + 6) * ma1(i + 6, 1) * (1 - ttt(i)) + n(i + 6) * (1 - ma1(i + 6, 1)), H(3));
        end
        for i = 1:3
            mon = mon + H(i) * ttt(i + 8) * ma2(i, 3) + H(i) * ma2(i, 2) + H(i) * ttt(i + 8) * ma2(i,
1) * ma2(i, 4) * ttt(i + 11);
        end
        for i = 1:3
            D = min(H(i) * ma2(i, 1) * (1 - ttt(i + 8)) + H(i) * (1 - ma2(i, 1)), D);
        end
        z = [ma2(1, 1) ma2(2, 1) ma2(3, 1) ma3(1)];
        for i = 1:4
            beta = beta + z(i);
            for j = i:4
                beta = beta - z(i) * z(j);
                for k = j:4

```

```

        beta = beta + z(i) * z(j) * z(k);
        for kk = k:4
            beta = beta - z(i) * z(j) * z(k) * z(kk);
        end
    end
end
end
mon = mon + D * ma3(2) + D * ttt(14) * ma3(3) + D * (1 - ttt(14)) * 40 * beta + D *
beta * ttt(15) * 40;
nn = D * beta * ttt(15);
n = nn * n / 100;
inco = inco + D * ttt(14) * (1 - beta) + D * (1 - ttt(14)) * beta;
end
end

```

问题四

问题三更新求解(MATBAL)

```

%% 代码说明
% 求解第四问-问题三情况下的代码
% 训练鲸鱼算法
% 鲸鱼数量: 10
% 最大迭代数为: 50
% 决策变量的维度为: 4
%% 主程序
clc
clear
jingyushu = 10; % 鲸鱼数量
MAXGEN = 100; % 最大迭代次数
weidu = 16; % 决策变量的维度
ub = 1 * ones(1, weidu); % 决策变量的上界
lb = 0 * ones(1, weidu); % 决策变量的下界
L_p = zeros(1, weidu); % 最优解的位置
id = 1; % 情况编号
L_s = inf; % 最优解的适应度值
PS = rand(jingyushu, weidu) .* repmat(ub - lb, jingyushu, 1) + repmat(lb, jingyushu, 1); %
初始化鲸鱼位置
trace = zeros(1, MAXGEN); % 记录每一代的最优适应度值

% 定义 n 的范围
n_values = 100:100:1000;

% 初始化结果表格
results = cell(length(n_values), 19);
row_index = 1;

for n = n_values

```

```

% 重置最优解
L_p = zeros(1, weidu);
L_s = inf;

for kk = 1:MAXGEN
    % 更新参数
    a = 2 - kk * (2 / MAXGEN); % 线性递减的参数a
    a2 = -1 + kk * (1 / MAXGEN); % 线性递减的参数a2

    for i = 1:jingyushu
        % 边界处理：越界赋值为边界值
        ubb = PS(i, :) > ub;
        lbb = PS(i, :) < lb;
        PS(i, :) = (PS(i, :).*(~(ubb + lbb))) + ub.*ubb + lb.*lbb;

        % 计算适应度值
        fitness = ObjFun(PS(i, :), id, n);

        % 更新最优解
        if fitness < L_s
            L_s = fitness;
            L_p = PS(i, :);
        end
    end

    % 更新鲸鱼位置
    for i = 1:jingyushu
        r1 = rand(); r2 = rand();
        A = 2 * a * r1 - a; % 参数A
        C = 2 * r2; % 参数C
        b = 1; % 螺旋参数
        l = (a2 - 1) * rand() + 1; % 螺旋参数
        p = rand(); % 随机选择更新方式

        for j = 1:weidu
            if p < 0.5 % 包围捕食
                if abs(A) < 1
                    D = abs(C * L_p(j) - PS(i, j));
                    PS(i, j) = L_p(j) - A * D;
                else
                    rand_leader_index = floor(jingyushu * rand() + 1);
                    X_rand = PS(rand_leader_index, :);
                    D_X_rand = abs(C * X_rand(j) - PS(i, j));
                    PS(i, j) = X_rand(j) - A * D_X_rand;
                end
            else % 螺旋更新
                distance_LLeader = abs(L_p(j) - PS(i, j));
                PS(i, j) = distance_LLeader * exp(b * l) * cos(l * 2 * pi) + L_p(j);
            end
        end
    end
end

```

```

        end
    end
end

% 记录当前代的最优适应度值
trace(kk) = L_s;
end

% 结果输出
name = ["零件1检测率", "零件2检测率", "零件3检测率", "零件4检测率", "零件5检测率", "零件6检测率", "零件7检测率", "零件8检测率", "半成品1检测率", "半成品2检测率", "半成品3检测率", "半成品1拆解率", "半成品2拆解率", "半成品3拆解率", "成品检测率", "成品拆解率"];
[ans1, ans2] = ObjFun1(L_p, id, n);

% 保存结果到表格
results{row_index, 1} = n;
for col_index = 1:16
    results{row_index, col_index + 1} = L_p(col_index);
end
results{row_index, 18} = ans1;
results{row_index, 19} = ans2 * 200;

row_index = row_index + 1;
end

% 显示结果表格
results_table = cell2table(results, 'VariableNames', ['n', name, '成本', '收入']);
disp(results_table);

%% 模拟题目中的步骤1、2
function [mon, inco] = ObjFun1(ttt, id, n)
    inco = 0;
    mon = 0;
    ma1 = [0.11538 2 1; 0.21154 8 1; 0.17308 12 2; 0.076923 2 1; 0.15385 8 1; 0.057692 12 2; 0.13462 8 1; 0.096154 12 2];
    ma2 = [0.13462 8 4 6; 0.19231 8 4 6; 0.11538 8 4 6];
    ma3 = [0.15385 8 6 10];
    for zzz = 1:2
        H = [1000 1000 1000];
        D = 1000;
        beta = 0;
        for i = 1:8
            mon = mon + ma1(i, 2) * ma1(i, 3) + ttt(i) * ma1(i, 1) * ma1(i, 3);
        end
        for i = 1:3
            H(1) = min(n * ma1(i, 1) * (1 - ttt(i)) + n * (1 - ma1(i, 1)), H(1));
        end
    end
end

```

```

    for i = 1:3
        H(2) = min(n * ma1(i + 3, 1) * (1 - ttt(i)) + n * (1 - ma1(i + 3, 1)), H(2));
    end
    for i = 1:2
        H(3) = min(n * ma1(i + 6, 1) * (1 - ttt(i)) + n * (1 - ma1(i + 6, 1)), H(3));
    end
    for i = 1:3
        mon = mon + H(i) * ttt(i + 8) * ma2(i, 3) + H(i) * ma2(i, 2) + H(i) * ttt(i + 8) * ma2(i,
1) * ma2(i, 4) * ttt(i + 11);
    end
    for i = 1:3
        D = min(H(i) * ma2(i, 1) * (1 - ttt(i + 8)) + H(i) * (1 - ma2(i, 1)), D);
    end
    z = [ma2(1, 1) ma2(2, 1) ma2(3, 1) ma3(1)];
    for i = 1:4
        beta = beta + z(i);
        for j = i:4
            beta = beta - z(i) * z(j);
            for k = j:4
                beta = beta + z(i) * z(j) * z(k);
                for kk = k:4
                    beta = beta - z(i) * z(j) * z(k) * z(kk);
                end
            end
        end
    end
    mon = mon + D * ma3(2) + D * ttt(14) * ma3(3) + D * (1 - ttt(14)) * 40 * beta + D *
beta * ttt(15) * 40;
    nn = D * beta * ttt(15);
    n = nn * n / 100;
    inco = inco + D * ttt(14) * (1 - beta) + D * (1 - ttt(14)) * beta;
end
end

%% 模拟题目中的步骤3、4
function dnss = ObjFun(ttt, id, n)
    inco = 0;
    mon = 0;
    ma1 = [0.11538 2 1; 0.21154 8 1; 0.17308 12 2; 0.076923 2 1; 0.15385 8 1; 0.057692 12
2; 0.13462 8 1; 0.096154 12 2];
    ma2 = [0.13462 8 4 6; 0.19231 8 4 6; 0.11538 8 4 6];
    ma3 = [0.15385 8 6 10];
    for zzz = 1:2
        H = [1000 1000 1000];
        D = 1000;
        beta = 0;
        for i = 1:8
            mon = mon + ma1(i, 2) * ma1(i, 3) + ttt(i) * ma1(i, 1) * ma1(i, 3);
        end
    end

```

```

for i = 1:3
    H(1) = min(n * ma1(i, 1) * (1 - ttt(i)) + n * (1 - ma1(i, 1)), H(1));
end
for i = 1:3
    H(2) = min(n * ma1(i + 3, 1) * (1 - ttt(i)) + n * (1 - ma1(i + 3, 1)), H(2));
end
for i = 1:2
    H(3) = min(n * ma1(i + 6, 1) * (1 - ttt(i)) + n * (1 - ma1(i + 6, 1)), H(3));
end
for i = 1:3
    mon = mon + H(i) * ttt(i + 8) * ma2(i, 3) + H(i) * ma2(i, 2) + H(i) * ttt(i + 8) * ma2(i,
1) * ma2(i, 4) * ttt(i + 11);
end
for i = 1:3
    D = min(H(i) * ma2(i, 1) * (1 - ttt(i + 8)) + H(i) * (1 - ma2(i, 1)), D);
end
z = [ma2(1, 1) ma2(2, 1) ma2(3, 1) ma3(1)];
for i = 1:4
    beta = beta + z(i);
    for j = i:4
        beta = beta - z(i) * z(j);
        for k = j:4
            beta = beta + z(i) * z(j) * z(k);
            for kk = k:4
                beta = beta - z(i) * z(j) * z(k) * z(kk);
            end
        end
    end
end
mon = mon + D * ma3(2) + D * ttt(14) * ma3(3) + D * (1 - ttt(14)) * 40 * beta + D *
beta * ttt(15) * 40;
nn = D * beta * ttt(15);
n = nn * n / 100;
inco = inco + D * ttt(14) * (1 - beta) + D * (1 - ttt(14)) * beta;
end
anss = mon - inco * 200;
end

```